

Post-Training HandBook

Last Update: 2026-09-24

Contents

1	Introduction	1
1.1	Background Knowledge	1
1.1.1	Markov Decision Process	1
1.1.2	Useful Facts	1
1.1.3	Glossaries	2
1.2	Policy Gradient	2
1.2.1	Policy Gradient Estimator	2
1.2.2	REINFORCE algorithm	3
1.2.3	Actor-Critic Algorithm	3
I	Reinforcement Learning	5
2	RL for LLMs	6
2.1	Proximal Policy Optimization	6
2.1.1	PPO introduced by OpenAI	6
2.1.2	PPO in the context of RLHF	7
2.2	REINFORCE++	8
2.3	Implicit Reward in Direct Preference Optimization	9
2.3.1	DPO Objective	10
2.3.2	Reward Hacking in DPO	10
2.3.3	Common Failure Modes of DPO	10
2.3.4	Mitigating Reward Hacking and Over-Optimization in DPO	11
2.4	Group Relative Policy Optimization	11
2.5	Group Sequence Policy Optimization	12
2.6	Clipped Importance Sampling Policy Optimization	13
2.6.1	Gradient Difference between PPO and CISPO	14
3	Agentic RL	15
3.1	Context Compression	15
3.1.1	Motivation	15
3.1.2	Compaction Mechanism	15
3.1.3	CompactionRL: PPO with Cross-Segment GAE	16
3.1.4	SUPO: GRPO with Rollout-Level Group Advantages	17
3.2	Asynchronous Algorithms	18
3.2.1	Introduction	18
3.2.2	SAO: Single-Rollout Asynchronous Optimization for Agentic Reinforcement Learning	20
4	Reward Modeling	27
4.1	Reward Modeling	27
4.1.1	Bradley–Terry Preference Reward Model	27
4.1.2	Outcome Reward Models	28
4.1.3	Process Reward Models	29
4.1.4	Reward Models versus Value Functions	30
4.1.5	Inference Interfaces	30
5	Distillation	31
5.1	On-Policy Distillation: Basics	33
5.1.1	Implemented OPD Objectives	34
5.1.2	Gradients of the OPD Objective	36
5.1.3	Pseudocode	38
5.2	OPD Implementation Discussions	39
5.2.1	Original VeRL	39
5.2.2	Rethink OPD modified VeRL	41

A Appendix	48
A.1 Glossary	48
A.1.1 Reward Model and Critic Model	48
A.1.2 How to train a Critic Model under the PPO framework	48

Chapter 1

Introduction

1.1 Background Knowledge

1.1.1 Markov Decision Process

Definition 1.1 (State, Action, Trajectory). A sequence of actions and states with T interactions with environment is defined as a trajectory $\tau = (s_0, a_0, s_1, a_1, s_2, a_2, \dots, a_{T-1}, s_{T-1})$

Definition 1.2 (Reward and Return). At time step t , the agent observes state s_t , takes action a_t , and transitions to the next state s_{t+1} . The single-step reward is determined by the transition and is defined as

$$r_t = R(s_t, a_t, s_{t+1}).$$

The (discounted) **reward** on the trajectory τ is defined as

$$R(\tau) = \sum_{t=0}^{T-1} \gamma^t r_t,$$

where $\gamma \in (0, 1]$ is the discount factor. The discounted **return** starting from time step t is defined as

$$G_t(\tau) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}.$$

Equivalently, G_t satisfies the recursive relation

$$G_t = r_t + \gamma G_{t+1}.$$

Definition 1.3 (State-Value Function and Value Function). The value function for the policy π is defined as

$$\begin{aligned} V^\pi(s_t) &= \mathbb{E}_\pi[G_t | s_t] = \mathbb{E}_\pi[r_t + \gamma G_{t+1} | s_t] = \mathbb{E}_{a_t \sim \pi(\cdot | s_t)} \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} [r_t + \gamma \mathbb{E}_\pi[G_{t+1} | s_{t+1}]] \\ &= \mathbb{E}_{a_t \sim \pi(\cdot | s_t)} \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} [r_t + \gamma V^\pi(s_{t+1})]. \end{aligned}$$

The state-value function for the policy π is defined as

$$Q^\pi(s_t, a_t) = \mathbb{E}_\pi[G_t | s_t, a_t] = \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} [r_t + \gamma V^\pi(s_{t+1})].$$

Therefore,

$$V^\pi(s_t) = \mathbb{E}_{a_t \sim \pi(\cdot | s_t)} [Q^\pi(s_t, a_t)]$$

Definition 1.4 (Advantage). The advantage function, by its definition, measures whether or not the action is better or worse than the policy's default behavior.

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t) = \mathbb{E}_{s_{t+1} \sim P(\cdot | s_t, a_t)} \left[\underbrace{r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)}_{\text{temporal difference}} \right] \quad (1.1)$$

1.1.2 Useful Facts

1. Probability of a trajectory

$$P(\tau | \theta) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} | s_t, a_t) \pi_\theta(a_t | s_t) \quad (1.2)$$

2. log derivative trick

$$\nabla_\theta P(\tau | \theta) = P(\tau | \theta) \nabla_\theta \log P(\tau | \theta) \quad (1.3)$$

3. expected Grad-Log-Prob Lemma

$$\mathbb{E}_{x \sim p(\cdot; \theta)} [\nabla_\theta \log p(x; \theta)] = 0 \quad (1.4)$$

1.1.3 Glossaries

Definition 1.5 (Estimation of the KL divergence). There are different estimators of the KL divergence, which can be found [here](#)². Recall that $\text{KL}(q||p) = E_{x \sim q} \left[\log \frac{q(x)}{p(x)} \right]$,

- k_1 estimator: $\frac{1}{n} \sum_{i=1}^n \log \frac{q(x_i)}{p(x_i)}$. Unbiased, high variance. Can be negative.
- k_2 estimator: $\frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left(\log \frac{p(x_i)}{q(x_i)} \right)^2$. biased, low variance. Always positive.
- k_3 estimator: $\frac{p(x)}{q(x)} - 1 - \log \frac{p(x)}{q(x)}$. Always positive. Better than k_2 ? (non-negativity: $\log a \leq a - 1$ for all $a > 0$).

vd: To add: comparison with three estimators

1.2 Policy Gradient

The goal of the policy-based RL is to maximize the expected reward

$$\max \left\{ \mathcal{J}(\theta) := \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] = \int P(\tau|\theta) R(\tau) d\tau \right\}, \quad (1.5)$$

where $P(\tau|\theta)$ is defined in (1.2) and $s_0 \sim \rho_0(\cdot)$

- $a_t \sim \pi_\theta(\cdot|s_t)$ for all $t \in \{0, 1, \dots, T-1\}$.
- $s_{t+1} \sim P(\cdot|s_t, a_t)$ for all $t \in \{0, 1, \dots, T-1\}$.
- $r_t = R(s_t, a_t, s_{t+1})$ for all $t \in \{0, 1, \dots, T-1\}$.

Applying (1.2) and (1.3), we get

$$\begin{aligned} \nabla_\theta \mathcal{J}(\theta) &= \int \nabla_\theta P(\tau|\theta) R(\tau) d\tau = \int P(\tau|\theta) \nabla_\theta \log P(\tau|\theta) R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log P(\tau|\theta) R(\tau)] \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\nabla_\theta \left(\rho_0(s_0) + \sum_{t=0}^{T-1} P(s_{t+1}|s_t, a_t) \right) + \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right] R(\tau) \\ &= \mathbb{E}_{\tau \sim \pi_\theta} \left[R(\tau) \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t|s_t) \right] \end{aligned} \quad (1.6)$$

1.2.1 Policy Gradient Estimator

Given N trajectories $\{\tau_1, \dots, \tau_N\}$ with $\tau_n \sim \pi_\theta$ for all $n \in [N]$, how can we construct a "good" policy gradient estimator?

The natural choice would be

- Naive unbiased estimator

$$\widetilde{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} R(\tau_n) \nabla_\theta \log \pi_\theta(a_t|s_t) \right)$$

However, the above looks a bit weird in the sense that $\log \pi_\theta(a_t|s_t)$ contains single-step probability while $R(\tau_n)$ represents the whole trajectory. This naturally leads to the following abstraction

$$\widetilde{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} \Phi_t \nabla_\theta \log \pi_\theta(a_t|s_t) \right)$$

Some popular choices of Φ_t (Schulman et al., 2015), hence the gradient estimator can be

- Reward-to-go unbiased estimator (drops a_t 's dependency on the past actions)

$$\widetilde{\nabla_\theta \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} G_t(\tau_n) \nabla_\theta \log \pi_\theta(a_t|s_t) \right)$$

Recall $G_t(\tau) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$. If one adopts this estimator, then the algorithm is called **REINFORCE** algorithm. This method drops the need on the Value function (see details below) but suffers from the high variance in gradient estimates.

- with baseline (variance reduction)

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} (G_t(\tau_n) - b(s_t)) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right)$$

One popular choice of the baseline function is $b(s_t) := V^{\pi}(s_t)$. When V^{π} is unknown, we can approximate it with $V_{\phi}^{\pi}(\cdot)$, a neural network parameterized by ϕ .

- Advantages based estimator

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} A^{\pi}(s_t, a_t) \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right),$$

where $A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t)$.

- TD-error based estimator

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left((r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)) \sum_{t=0}^{T_n-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right),$$

where $r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$ is a approximation of the $A^{\pi}(s_t, a_t)$.

- GAE(γ, λ) estimator

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{\ell=0}^{T_n} (\gamma \lambda)^{\ell} \delta_{t+\ell} \sum_{t=0}^{T_n-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right),$$

where $\delta_t = r_t + \gamma V_{\phi}^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t)$. GAE($\gamma, 0$) typically has low variance and GAE($\gamma, 1$) has high variance.

1.2.2 REINFORCE algorithm

If one adopts the Reward-to-go unbiased estimator, we can reach to the REINFORCE algorithm. The objective function is

$$\mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left(\sum_{t=0}^{T-1} G_t(\tau) \log \pi_{\theta}(a_t | s_t) \right) \quad (1.7)$$

where $G_t(\tau) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}$. This method drops the need on the Value function but suffers from the high variance in gradient estimates. The algorithm is summarized in Algorithm 1

Algorithm 1 REINFORCE

- 1: Inputs: Initial Policy network parameter θ_0 and stepsize α .
- 2: **for** $k=0,1,2,\dots$ **do**
- 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
- 4: Reward-to-go calculation $G_t(\tau_n) = \sum_{t'=t}^{T_n-1} \gamma^{t'-t} r_{t'}$ for all n .
- 5: Compute the gradient estimate

$$\widetilde{\nabla_{\theta} \mathcal{J}(\theta_k)} = \frac{1}{N} \sum_{n=1}^N \left(\sum_{t=0}^{T_n-1} G_t(\tau_n) \nabla_{\theta} \log \pi_{\theta_k}(a_t | s_t) \right)$$

- 6: Policy update $\theta_{k+1} \leftarrow \theta_k + \alpha \widetilde{\nabla_{\theta} \mathcal{J}(\theta_k)}$
 - 7: **end for**
-

1.2.3 Actor-Critic Algorithm

When one adopts the estimator for the policy gradient, usually the state value function is used. Let us approximate it with V_{ϕ}^{π} the objective function for TD-residual based gradient estimator can written as

$$\max \left\{ \mathcal{J}_{\text{TD}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta}} \left[\sum_{t=0}^{T-1} (r_t + \gamma V_{\phi}^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t)) \log \pi_{\theta}(a_t | s_t) \right] \right\}.$$

Similarly, the V_{ϕ}^{π} can be updated by

$$\min \left\{ \mathcal{L}(\phi) := \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} [(r_t + \gamma V_\phi^\pi(s_{t+1}) - V_\phi^\pi(s_t))^2] \right] \right\}. \quad (1.8)$$

This is actually the actor-critic framework. When we drive $\mathcal{L}(\phi)$ to 0, intuitively, the advantage will be close to 0, which means that for the actor, π_θ is approaching the optimal policy and the value function is also approaching the optimal.

vd: To add

Algorithm 2 Naive Actor-Critic Algorithm

- 1: Inputs: Initial Policy network parameter θ_0 and stepsize α .
 - 2: **for** $k=0,1,2,\dots$ **do**
 - 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
 - 4: Reward-to-go calculation $G_t(\tau_n) = \sum_{t'=t}^{T_n-1} \gamma^{t'-t} r'_{t'}$ for all n .
 - 5: Compute the gradient estimate
 - ...
 - 6: Policy update $\theta_{k+1} \leftarrow \theta_k + \alpha \widetilde{\nabla_\theta \mathcal{J}(\theta_k)}$
 - 7: **end for**
-

Part I

Reinforcement Learning

Chapter 2

RL for LLMs

2.1 Proximal Policy Optimization

2.1.1 PPO introduced by OpenAI

When performing the policy gradient update, each step one needs to sample the trajectories of the environment (since after one update on the policy π_θ , the trajectory becomes obsolete. How can we address the sample efficiency issue? Importance sampling comes to rescue.

Consider the gradient estimator function, where $\hat{A}_\phi$ is any "good" estimate of the advantage¹.

$$\begin{aligned}\widetilde{\nabla_\theta \mathcal{J}}(\theta) &= \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \hat{A}_\phi(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t) \nabla_\theta \log \pi_\theta(a_t | s_t) \right] \\ &= \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{1}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t) \nabla_\theta \pi_\theta(a_t | s_t) \right].\end{aligned}$$

The corresponding objective function can be written as

$$\mathcal{J}_{\text{advantage}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t) \right]$$

In practice, we cannot sample enough trajectories, hence the estimation of the $\mathcal{J}_{\text{advantage}}(\theta)$ through sample average is not accurate. This will be worsen by the fact that if $\pi_{\theta_{\text{old}}}$ is very different from π_θ . This leads to two versions of the PPO objective functions.

$$\max_{\theta} \mathcal{J}_{\text{PPO-clip}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \min \left\{ \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_\phi(s_t, a_t), \text{clip} \left(\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_\phi(s_t, a_t) \right\} \right], \quad (2.1)$$

where

$$\text{clip}(x, 1 - \epsilon, 1 + \epsilon) = \begin{cases} x, & \text{if } 1 - \epsilon < x < 1 + \epsilon \\ 1 - \epsilon, & \text{if } x \leq 1 - \epsilon \\ 1 + \epsilon, & \text{if } x \geq 1 + \epsilon. \end{cases}$$

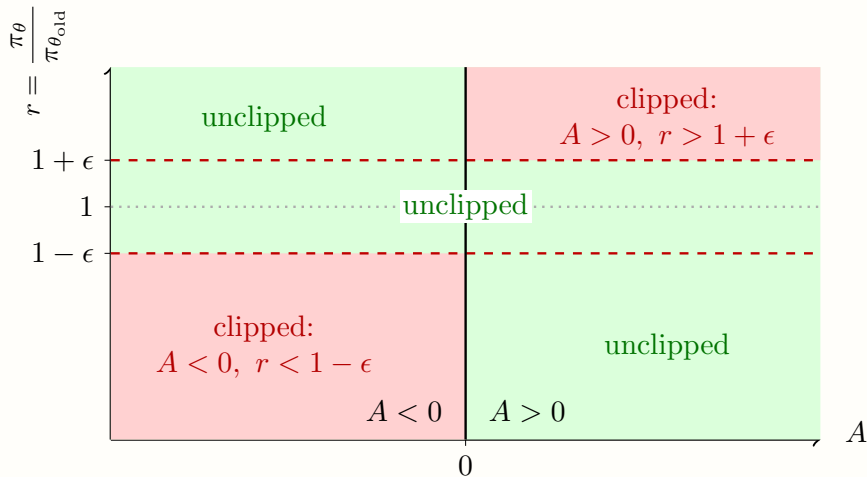


Figure 2.1: PPO-Clip trust region in the (A, r) plane, where $r = \pi_\theta(a|s)/\pi_{\theta_{\text{old}}}(a|s)$. The red region is the clipped region, and the green region is the unclipped region (after the clip and min operation).

¹As the subscripts suggests, the definition of $\hat{A}_\phi$ usually rely on the value function V_ϕ^π .

Remark

- The popular choice of $\hat{A}_\phi(s_t, a_t)$ is $A_\phi^{\text{GAE}(\gamma, \lambda)} = \sum_{\ell=0}^{T-1-t} (\gamma\lambda)^\ell \left(r_{t+\ell} + \gamma V_\phi^\pi(s_{t+1+\ell}) - V_\phi^\pi(s_{t+\ell}) \right)$.
- When $\hat{A}_\phi(s_t, a_t) > 0$, which means that a_t is better than the average, we want to increase its probability. However, we need to limit the increase to not being greater than the factor of $1 + \epsilon$.
- When $\hat{A}_\phi(s_t, a_t) < 0$, which means that a_t is worse than the average, we want to decrease its probability. However, we need to limit the decrease to be no greater than the factor of $1 - \epsilon$.

One can also constraint the distribution shift by

$$\max_{\theta} \mathcal{J}_{\text{PPO-KL}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_\phi(s_t, a_t) - \beta \text{KL} [\pi_{\theta_{\text{old}}}(\cdot|s_t) || \pi_{\theta}(\cdot|s_t)] \right], \quad (2.2)$$

Remark

- When maximize $\mathcal{J}_{\text{PPO-KL}}(\theta)$, we minimize $\mathbb{D}_{\text{KL}} [\pi_{\theta_{\text{old}}}(\cdot|s_t) || \pi_{\theta}(\cdot|s_t)]$, i.e., we minimize the forward KL, which has the [mean-seeking behavior](#)[Ⓔ].

To put pieces together, we summarize the PPO-Clip in Algorithm 3.

Algorithm 3 PPO-Clip Algorithm

-
- 1: Inputs: Initial Policy network parameter θ_0 and value function parameter ϕ_0 .
 - 2: **for** $k=0,1,2,\dots$ **do**
 - 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
 - 4: Reward-to-go calculation $G_t(\tau_n) = \sum_{t'=t}^{T_n-1} \gamma^{t'-t} r_{t'}$ for all n .
 - 5: GAE calculation $A_{\phi_k}^{\text{GAE}(\gamma, \lambda)}(s_t^{(n)}, a_t^{(n)})$ for all t and n ².
 - 6: Approximately solve

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_{\theta}(a_t^{(n)}|s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)}|s_t^{(n)})} A_{\phi_k}^{\text{GAE}(\gamma, \lambda)}(s_t^{(n)}, a_t^{(n)}), \right. \\ \left. \text{clip} \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) A_{\phi_k}^{\text{GAE}(\gamma, \lambda)}(s_t^{(n)}, a_t^{(n)}) \right\},$$

- 7: Approximately solve

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} (V_{\phi}^{\pi}(s_t^{(n)}) - G_t(\tau_n))^2$$

- 8: **end for**
-

2.1.2 PPO in the context of RLHF

Let us first introduce a new set of notions and setup. Let $\mathcal{D}$ be a set of prompt and $x \in \mathcal{D}$ is any sample from it. $y \sim \pi_{\theta}(\cdot|x)$ be a response sampled from the policy model π_{θ} given the prompt x . To simplify the discussion, let us assume that the reward model is given and we focus on the outcome reward $r(x, y)$.

We consider the following variant objective function inspired by policy gradient and PPO

$$\mathcal{J}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta}(\cdot|x)} [r(x, y) - \beta \text{KL} [\pi_{\theta}(\cdot|x) || \pi_{\theta_{\text{ref}}}(\cdot|x)]],$$

note that here we used forward KL divergence. Then the gradient is

$$\nabla \mathcal{J}(\theta) = \int_{x \in \mathcal{X}} p(x) \left\{ \left[\int_{y \in \mathcal{Y}(x)} r(x, y) \frac{\partial}{\partial \theta} \pi_{\theta}(y|x) dy \right] - \beta \left[\int_{s \in \mathcal{Y}(x)} \frac{\partial}{\partial \theta} \pi_{\theta}(s|x) \log \frac{\pi_{\theta}(s|x)}{\pi_{\theta_{\text{ref}}}(y|x)} ds \right] \right\} dx, \quad (2.3)$$

where $\mathcal{X} = \mathcal{D}$ and $\mathcal{Y}(x) = \text{support}(\pi_\theta(y|x))$. Note that

$$\begin{aligned}
& \int_{s \in \mathcal{Y}(x)} \frac{\partial}{\partial \theta} \pi_\theta(s|x) \log \frac{\pi_\theta(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} ds \\
&= \int_{s \in \mathcal{Y}(x)} \left[\nabla \pi_\theta(s|x) \cdot \log \frac{\pi_\theta(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} + \pi_\theta(s|x) \cdot \nabla \log \frac{\pi_\theta(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} \right] ds \\
&= \int_{s \in \mathcal{Y}(x)} \left[\nabla \pi_\theta(s|x) \cdot \log \frac{\pi_\theta(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} + \nabla \pi_\theta(s|x) \right] ds \\
&= \int_{s \in \mathcal{Y}(x)} \nabla \pi_\theta(s|x) \cdot \log \frac{\pi_\theta(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} ds
\end{aligned} \tag{2.4}$$

where the last line follows from (1.4). Combining (2.3) and (2.4), one obtains

$$\begin{aligned}
\nabla \mathcal{J}(\theta) &= \int_{x \in \mathcal{X}} p(x) \left\{ \left[\int_{y \in \mathcal{Y}(x)} r(x, y) \frac{\partial}{\partial \theta} \pi_\theta(y|x) dy \right] - \beta \left[\int_{s \in \mathcal{Y}(x)} \nabla \pi_\theta(s|x) \cdot \log \frac{\pi_\theta(s|x)}{\pi_{\theta_{\text{ref}}}(s|x)} ds \right] \right\} dx \\
&= \int_{x \in \mathcal{X}, y \in \mathcal{Y}(x)} p(x) \left\{ \pi_\theta(y|x) \left[r(x, y) \nabla \log \pi_\theta(y|x) - \beta \nabla \log \pi_\theta(y|x) \cdot \log \frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{ref}}}(y|x)} \right] \right\} dy dx \\
&= \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} \left[\left(r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{ref}}}(y|x)} \right) \nabla \log \pi_\theta(y|x) \right]
\end{aligned} \tag{2.5}$$

From (2.5), we conclude the "PPO" version of the RLHF objective function

$$\max_{\theta} \mathcal{J}_{\text{RLHF}}(\theta) := \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x)} \left[r(x, y) - \beta \log \frac{\pi_\theta(y|x)}{\pi_{\theta_{\text{ref}}}(y|x)} \right], \tag{2.6}$$

which recovers the first part of the objective function in Equation (2) of Ouyang et al. (2022).

2.2 REINFORCE++

REINFORCE++ modifies PPO in the LLM text by introducing

- token-level KL penalty $\text{KL}(t; \theta) = \log \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{ref}}}(a_t|s_t)}$, where $\pi_{\theta_{\text{ref}}}$ can be the SFT model;
- token-level reward function $r_t = \mathbb{I}(s_t = [\text{EOS}])r(x, y) - \beta \text{KL}(t)$, where x and y are input prompt and response, respectively and $r(x, y)$ is the outcome reward.

Recall the advantage used in PPO is $A_\phi^{\text{GAE}(\gamma, \lambda)} = \sum_{\ell=0}^{T-1-t} (\gamma\lambda)^\ell \left(r_{t+\ell} + \gamma V_\phi^\pi(s_{t+1+\ell}) - V_\phi^\pi(s_{t+\ell}) \right)$. If we drop the value network and plug in the new definition of r_t , we get

$$\begin{aligned}
A_t^{\text{GAE}(\gamma, \lambda)} &= \sum_{\ell=0}^{T-1-t} (\gamma\lambda)^\ell [\mathbb{I}(s_{t+\ell} = [\text{EOS}])r(x, y) - \beta \text{KL}(t + \ell; \theta)] \\
&= \sum_{i=t}^{T-1} (\gamma\lambda)^\ell [\mathbb{I}(s_i = [\text{EOS}])r(x, y) - \beta \text{KL}(i)] = (\gamma\lambda)^{T-1} r(x, y) - \sum_{i=t}^{T-1} (\gamma\lambda)^i [\beta \text{KL}(i; \theta)].
\end{aligned}$$

Specifically, letting $\gamma = \lambda = 1$, we get the advantage estimation used in REINFORCE++ as

$$A_t^{\text{RF}++} = r(x, y) - \sum_{i=t}^{T-1} \beta \text{KL}(i; \theta). \tag{2.7}$$

Consequently, for the k th episode, REINFORCE++ solves the following problem

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_\theta(a_t^{(n)}|s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)}|s_t^{(n)})} \bar{A}_{n,t}, \text{clip} \left(\frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \bar{A}_{n,t} \right\},$$

where $\bar{A}_{n,t} = \frac{A_{n,t}^{\text{RF}++} - \mu_k}{\sigma_k}$, $\mu_k = \text{mean}\{A_{n,t}^{\text{RF}++}\}_{n \in [1, N], t \in [0, T_n-1]}$, $\sigma_k = \text{std}\{A_{n,t}^{\text{RF}++}\}_{n \in [1, N], t \in [0, T_n-1]}$, $A_{n,t}^{\text{RF}++} = r(x^{(n)}, y^{(n)}) - \sum_{i=t}^{T_n-1} \beta \text{KL}(i; \theta_k)$, and $(x^{(n)}, y^{(n)})$ is the n th sample.

To conclude, we summarize REINFORCE++ in Algorithm 4.

Algorithm 4 REINFORCE++

-
- 1: Inputs: Initial Policy network parameter θ_0 .
 - 2: **for** $k=0,1,2,\dots$ **do**
 - 3: Collect a set of trajectories $D_k = \{\tau_n\}_{n=1}^N$.
 - 4: $\{A_{n,t}^{\text{RF++}}\}_{n \in [1,N], t \in [0, T_n-1]}$ calculation, batch mean μ_k and variance σ_k calculation.
 - 5: Compute normalized advantage estimation $\{\bar{A}_{n,t} = \frac{A_{n,t}^{\text{RF++}} - \mu_k}{\sigma_k}\}_{n \in [1,N], t \in [0, T_n-1]}$.
 - 6: Approximately solve
-

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_{\theta}(a_t^{(n)} | s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)} | s_t^{(n)})} \bar{A}_{n,t}, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \bar{A}_{n,t} \right\},$$

7: **end for**

Remark

- Compared to GRPO, the normalization happens on the "batch" level; while GRPO applies the normalization within the "group" level.
- The KL penalty applied in the loss function is different. For GRPO, the KL penalty is taken out of the definition of the advantage estimation and uses the k_3 estimator (recall the definition in Definition 1.5), which is $\beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{\text{ref}}](i, t)$. In REINFORCE++, the KL penalty is factorized in the definition of the advantage estimation and uses the k_1 estimator, which is $\beta \sum_{i=t}^{T-1} \text{KL}(i; \theta)$. Notice that in the t th action, the penalty is the accumulation of all KL penalties starting from the time step t to the end.

Motivated by the discussion above, Xie et al. (2025) modifies the loss function by combining the GRPO and REINFORCE++ as below.

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{N} \sum_{n=1}^N \frac{1}{T_n} \sum_{t=0}^{T_n-1} \min \left\{ \frac{\pi_{\theta}(a_t^{(n)} | s_t^{(n)})}{\pi_{\theta_k}(a_t^{(n)} | s_t^{(n)})} \bar{A}_{n,t}, \text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_k}(a_t | s_t)}, 1 - \epsilon, 1 + \epsilon \right) \bar{A}_{n,t} - \beta \mathbb{D}_{KL}[\pi_{\theta} || \pi_{\text{ref}}](i, t) \right\},$$

where $\bar{A}_{n,t} = \frac{A_{n,t}^{\text{Logic}} - \mu_k}{\sigma_k}$, $\mu_k = \text{mean}\{A_{n,t}^{\text{Logic}}\}_{n \in [1,N], t \in [0, T_n-1]}$, $\sigma_k = \text{std}\{A_{n,t}^{\text{Logic}}\}_{n \in [1,N], t \in [0, T_n-1]}$, $A_{n,t}^{\text{Logic}} = r(x^{(n)}, y^{(n)})$. Basically, it moves KL penalty out of the definition of the advantage estimators and use k_3 KL estimation.

2.3 Implicit Reward in Direct Preference Optimization

Direct Preference Optimization (DPO) does not explicitly train a separate reward model. Instead, it implicitly defines a reward function through the relationship between the optimized policy and the reference policy. DPO can be derived from the KL-regularized RLHF objective:

$$\max_{\pi} \mathbb{E}_{y \sim \pi(\cdot | x)} [r(x, y)] - \beta D_{\text{KL}}(\pi(\cdot | x) || \pi_{\text{ref}}(\cdot | x)),$$

where π_{ref} is a fixed reference policy, usually the supervised fine-tuned model, and $\beta > 0$ controls the strength of the KL regularization.

The optimal policy of this objective satisfies

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp \left(\frac{1}{\beta} r(x, y) \right),$$

where $Z(x)$ is the partition function depending only on the prompt x .

Solving for the reward gives

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

Since $\beta \log Z(x)$ depends only on the prompt x , it cancels when comparing two responses to the same prompt. Therefore, DPO uses the following implicit reward parameterization:

$$r_{\theta}(x, y) = \beta \log \frac{\pi_{\theta}(y | x)}{\pi_{\text{ref}}(y | x)} + C(x),$$

where $C(x)$ is a prompt-dependent constant.

For a preferred response y_w and a rejected response y_l , the reward difference is

$$\begin{aligned} r_\theta(x, y_w) - r_\theta(x, y_l) &= \beta \left[\log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right] \\ &= \beta [\log \pi_\theta(y_w | x) - \log \pi_\theta(y_l | x) - \log \pi_{\text{ref}}(y_w | x) + \log \pi_{\text{ref}}(y_l | x)]. \end{aligned}$$

Thus, the implicit reward in DPO measures how much the optimized policy increases the probability of a response relative to the reference policy.

2.3.1 DPO Objective

DPO is trained on preference data of the form

$$(x, y_w, y_l) \sim \mathcal{D},$$

where x is the prompt, y_w is the preferred response, and y_l is the rejected response.

The DPO loss is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \left[\log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right] \right) \right],$$

where $\sigma(\cdot)$ is the sigmoid function.

Define

$$\Delta_\theta(x, y_w, y_l) = \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)}.$$

Then the DPO loss can be written compactly as

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} [\log \sigma(\beta \Delta_\theta(x, y_w, y_l))].$$

Minimizing this loss increases the relative log-probability of the preferred response over the rejected response, after correcting for their probabilities under the reference policy.

2.3.2 Reward Hacking in DPO

DPO does not use an explicit reward model during optimization. Therefore, it is less likely to suffer from the classical online reward hacking problem in PPO-style RLHF, where the policy repeatedly exploits weaknesses of a learned reward model.

However, DPO can still suffer from a different form of reward hacking or over-optimization. Since the implicit reward is determined by the preference dataset,

$$(x, y_w, y_l) \sim \mathcal{D},$$

DPO may exploit biases or artifacts in the preference data.

In this sense, DPO does not hack an explicit reward model. Instead, it may overfit or exploit the implicit preference signal encoded in the dataset:

DPO may hack the preference dataset rather than a learned reward model.

For example, if preferred responses are systematically longer than rejected responses, the model may learn the spurious rule

$$\text{longer response} \Rightarrow \text{better response}.$$

If preferred responses often contain certain stylistic patterns, the model may learn to imitate those patterns even when they do not improve semantic quality.

Thus, DPO can still exhibit reward-hacking-like behavior through preference-data bias, distribution shift, or over-optimization of the implicit reward.

2.3.3 Common Failure Modes of DPO

A common failure mode of DPO is length bias. The sequence log-probability is

$$\log \pi_\theta(y | x) = \sum_{t=1}^{|y|} \log \pi_\theta(y_t | x, y_{<t}).$$

If preferred responses are usually longer than rejected responses, the model may learn to favor longer responses, even when they are not more accurate or helpful.

Another failure mode is style bias. If preferred responses share superficial stylistic features, such as excessive politeness, rigid formatting, or repeated templates, DPO may learn these features as proxies for quality. DPO may also overfit to easy negatives. If the rejected response y_l is much worse than the preferred response y_w , the model only learns a coarse distinction:

$$y_w \gg y_l.$$

This may not teach the model how to distinguish between two high-quality responses.

Finally, DPO is an offline preference optimization method. It is trained on fixed preference data, but after training, the model generates responses from its own policy:

$$y \sim \pi_\theta(\cdot | x).$$

If this generation distribution differs significantly from the training preference data distribution, DPO may suffer from distribution shift.

2.3.4 Mitigating Reward Hacking and Over-Optimization in DPO

One important mitigation is improving the quality of the preference data. The preference dataset should minimize spurious correlations between preference labels and superficial features such as length, format, or style. Ideally, the preference label should reflect semantic quality rather than dataset artifacts.

To reduce length bias, one can use length-normalized log-probabilities. Instead of using

$$\log \pi_\theta(y | x) = \sum_{t=1}^{|y|} \log \pi_\theta(y_t | x, y_{<t}),$$

one may use the average log-probability

$$\frac{1}{|y|} \log \pi_\theta(y | x) = \frac{1}{|y|} \sum_{t=1}^{|y|} \log \pi_\theta(y_t | x, y_{<t}).$$

Another mitigation is tuning the temperature parameter β . The parameter β controls the strength of the preference margin in the DPO loss:

$$\log \sigma(\beta \Delta_\theta(x, y_w, y_l)).$$

If β is too large, the optimization may aggressively separate preferred and rejected responses, increasing the risk of over-optimization. A smaller β makes the update more conservative.

One can also monitor or explicitly regularize the divergence from the reference model:

$$D_{\text{KL}}(\pi_\theta || \pi_{\text{ref}}).$$

Although DPO already uses the reference policy in its objective, explicit KL monitoring or early stopping can help prevent excessive drift from the reference model.

Another practical method is to mix the DPO loss with a supervised fine-tuning loss:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{DPO}}(\theta) + \lambda_{\text{SFT}} \mathcal{L}_{\text{SFT}}(\theta),$$

where

$$\mathcal{L}_{\text{SFT}}(\theta) = -\mathbb{E}_{(x, y_w) \sim \mathcal{D}} [\log \pi_\theta(y_w | x)].$$

The SFT loss helps preserve language quality, instruction-following behavior, and formatting stability.

Finally, iterative preference data collection can reduce distribution shift. Instead of training only on a fixed offline dataset, one can repeatedly sample responses from the current policy,

$$y \sim \pi_\theta(\cdot | x),$$

collect new preference labels, and continue preference optimization on data closer to the current model distribution.

2.4 Group Relative Policy Optimization

GRPO is motivated by removing the requirements on estimating the value function is used PPO Algorithm 3. For each question $q \sim P(Q)$, GRPO samples a group of outputs $\{o_1, o_2, \dots, o_G\}$ from the old policy π_{old} .

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}_{[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\text{old}}(O|q)]}$$

$$\left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[\frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\text{old}}(o_{i,t}|q, o_{i,<t})} \hat{A}_{i,t}, \text{clip} \left(\frac{\pi_\theta(o_{i,t}|q, o_{i,<t})}{\pi_{\text{old}}(o_{i,t}|q, o_{i,<t})}, 1 - \varepsilon, 1 + \varepsilon \right) \hat{A}_{i,t} \right] - \beta \mathbb{D}_{\text{KL}}[\pi_\theta || \pi_{\text{ref}}](i, t) \right\} \right], \quad (2.8)$$

where

- $\mathbb{D}_{KL}[\pi_\theta || \pi_{\text{ref}}]$ is the k_3 estimator of the KL divergence with

$$\mathbb{D}_{KL}[\pi_\theta || \pi_{\text{ref}}](i, t) = \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_\theta(o_{i,t} | q, o_{i,<t})} - \log \frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_\theta(o_{i,t} | q, o_{i,<t})} - 1$$

- the advantage estimate is defined as $\hat{A}_{i,t} := \frac{r_i - \text{mean}(\mathbf{r})}{\text{std}(\mathbf{r})}$ with $\mathbf{r} = \{r_1, r_2, \dots, r_G\}$.

Remark

- The reverse KL divergence is used, which has the mode-seeking behavior. The usage of k_3 estimator is crucial for GRPO because k_3 guarantee to be non-negative. Otherwise, if one adopts the k_1 estimator, then there could be a possibility to "hack" the training process by making the k_1 estimator arbitrarily small (very negative), then the reward is $\mathcal{J}_{GRPO}(\theta)$ is also maimized. However, this means that π_θ is very different from π_{ref} . At the same time, numerical instability is observed during the training dynamics. Because the computation of $\exp\left(\frac{\pi_{\text{ref}}(o_{i,t} | q, o_{i,<t})}{\pi_\theta(o_{i,t} | q, o_{i,<t})}\right)$ can be unstable.
- In the computation of the KL term, note that $o_i \sim \pi_{\theta_{\text{old}}}$. But the link suggests that o_i should be sampled from π_θ to make $\mathbb{D}_{KL}[\pi_\theta || \pi_{\text{ref}}]$ an unbiased estimator. In practice, to make these make sense, one has to initialize $\pi_\theta := \pi_{\theta_{\text{old}}}$ and take only one step update.
- When optimize the objective function, one of tries to $\min_\theta -\mathcal{J}_{GRPO}(\theta)$. One should notice that $-\mathcal{J}_{GRPO}$ can be smaller than 0, meaning the loss can be negative. This could happen when "min" term is larger that the KL term, i.e., when either the two distributions are quite different or the advantage is large.

2.5 Group Sequence Policy Optimization

In GRPO, the token-level importance sampling weight $\frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}$ is applied.

GSPO argues that this is problematic since this weight is based on a single sample $o_{i,t}$ from each next-token distribution $\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})$, it fails to perform the intended distribution-correction role³. Instead, it introduces high-variance noise into the training gradients, which accumulates over long sequences and is exacerbated by the clipping mechanism.

Therefore, GSPO proposes to use the sequence-level importance weight $\frac{\pi_\theta(o_i | q)}{\pi_{\theta_{\text{old}}}(o_i | q)}$. Therefore, the loss function becomes (ignoring the KL regularization term)

$$\mathcal{J}_{GSPO}(\theta) = \mathbb{E}_{[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)]} \left[\frac{1}{G} \sum_{i=1}^G \min \left[s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_i \right] \right],$$

where $s_i(\theta) = \left(\frac{\pi_\theta(o_i | q)}{\pi_{\theta_{\text{old}}}(o_i | q)} \right)^{1/|o_i|} = \exp \left(\frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \log \left\{ \frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})} \right\} \right)$, and $\hat{A}_i = \frac{r(x, o_i) - \text{mean}(\{r(x, o_j)\}_{j=1}^G)}{\text{std}(\{r(x, o_j)\}_{j=1}^G)}$.

Therefore, GSPO applies clipping to entire responses instead of individual tokens to exclude the overly "off-policy" samples from gradient estimation, which matches both the sequence-level rewarding and optimization. Also, note that the clipping ranges in GSPO and in previous algorithms (e.g., GRPO) typically differ in order of magnitude due to the distinct definitions of importance ratios.

In scenarios like multi-turn RL, we may desire a finer-grained advantage adjustment than the sequence level.

$$\mathcal{J}_{GSPO\text{-token}}(\theta) = \mathbb{E}_{[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(O|q)]} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \left\{ \min \left[s_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(s_{i,t}(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_{i,t} \right] \right\} \right],$$

where $s_{i,t}(\theta) = \text{sg}[s_i(\theta)] \frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\text{sg}[\pi_\theta(o_{i,t} | q, o_{i,<t})]}$ and sg is the stop gradient operator, so numerically, $s_{i,t}(\theta) = s_i(\theta)$.

To see more clearly about the gradient of the GSPO, GSPO-token, and GRPO, we can compute (clipping is omitted for brevity)

$$\begin{aligned} \nabla_\theta \mathcal{J}_{GSPO}(\theta) &= \nabla_\theta \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G s_i(\theta) \hat{A}_i \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G s_i(\theta) \hat{A}_i \cdot \nabla_\theta \log s_i(\theta) \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \left(\frac{\pi_\theta(o_i | x)}{\pi_{\theta_{\text{old}}}(o_i | x)} \right)^{\frac{1}{|o_i|}} \hat{A}_i \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \nabla_\theta \log \pi_\theta(o_{i,t} | x, o_{i,<t}) \right] \end{aligned}$$

³Recall that Importance sampling needs multiple samples from the same distribution to reduce variance.

$$\begin{aligned}
\nabla_{\theta} \mathcal{J}_{\text{GSPO-token}}(\theta) &= \nabla_{\theta} \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} s_{i,t}(\theta) \hat{A}_{i,t} \right] \\
&= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G s_i(\theta) \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \hat{A}_{i,t} \frac{\nabla_{\theta} \pi_{\theta}(o_{i,t} | x, o_{i,<t})}{\pi_{\theta}(o_{i,t} | x, o_{i,<t})} \right] \\
&= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \left(\frac{\pi_{\theta}(o_i | x)}{\pi_{\theta_{\text{old}}}(o_i | x)} \right)^{\frac{1}{|o_i|}} \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \hat{A}_{i,t} \nabla_{\theta} \log \pi_{\theta}(o_{i,t} | x, o_{i,<t}) \right] \\
\nabla_{\theta} \mathcal{J}_{\text{GRPO}}(\theta) &= \nabla_{\theta} \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} w_{i,t}(\theta) \hat{A}_{i,t} \right] \\
&= \mathbb{E}_{x \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \hat{A}_i \cdot \frac{1}{|o_i|} \sum_{t=1}^{|o_i|} \frac{\pi_{\theta}(o_{i,t} | x, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | x, o_{i,<t})} \nabla_{\theta} \log \pi_{\theta}(o_{i,t} | x, o_{i,<t}) \right]
\end{aligned}$$

Therefore, the fundamental distinction between GSPO and GRPO lies in how they weight the gradients of the log likelihoods of tokens. In GRPO, the tokens are weighted according to their respective "importance weight". In contrast, GSPO weights all the tokens in a response equally, eliminating this instability factor of GRPO. GSPO-token and GSPO are numerically identical in the optimization objective, clipping condition, and theoretical gradient when we set the advantages of all the tokens in the response o_i to the same value (i.e., $\hat{A}_{i,t} = \hat{A}_i$), while GSPO-token enjoys the higher flexibility of adjusting the advantages per token.

$$r_{i,t} = \begin{cases} r_{i,t}^{\text{re}} + \gamma^{T_i-t} R_{\text{out},i}, & R_{\text{out},i} = +1, \\ \text{sim}_{i^*,t} \cdot r_{i^*,t}^{\text{re}} + \gamma^{T_i-t} R_{\text{out},i}, & R_{\text{out},i} = -1. \end{cases} \quad (2.9)$$

2.6 Clipped Importance Sampling Policy Optimization

Clipped Importance Sampling Policy Optimization (CISPO (Chen et al., 2025)) proposed in MiniMax-M1 optimizes a policy-gradient objective using a clipped importance sampling ratio as a stop-gradient weight. The CISPO objective is defined as

$$\mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E}_{(q,a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \log \pi_{\theta}(o_{i,t} | q, o_{i,<t}) \right].$$

Here, $\hat{A}_{i,t}$ is the estimated advantage for token $o_{i,t}$, and $\text{sg}(\cdot)$ denotes the stop-gradient operation. Therefore, $\text{sg}(\hat{r}_{i,t}(\theta))$ is treated as a constant when taking gradients with respect to θ . The clipped importance sampling ratio is

$$\hat{r}_{i,t}(\theta) = \text{clip}(r_{i,t}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}),$$

where the token-level importance sampling ratio is

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}.$$

Equivalently,

$$\hat{r}_{i,t}(\theta) = \begin{cases} 1 - \epsilon_{\text{low}}, & r_{i,t}(\theta) < 1 - \epsilon_{\text{low}}, \\ r_{i,t}(\theta), & 1 - \epsilon_{\text{low}} \leq r_{i,t}(\theta) \leq 1 + \epsilon_{\text{high}}, \\ 1 + \epsilon_{\text{high}}, & r_{i,t}(\theta) > 1 + \epsilon_{\text{high}}. \end{cases}$$

Thus, CISPO does not clip the surrogate objective itself. Instead, it clips the token-level importance sampling ratio and uses the clipped ratio as a stop-gradient coefficient in front of the log-probability term.

2.6.1 Gradient Difference between PPO and CISPO

The main difference between PPO and CISPO is how clipping affects the policy gradient. For PPO, the clipped surrogate objective at token (i, t) can be written as

$$L_{i,t}^{\text{PPO}}(\theta) = \min \left(r_{i,t}(\theta) \hat{A}_{i,t}, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{i,t} \right),$$

where

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}.$$

Consider the case

$$\hat{A}_{i,t} > 0 \quad \text{and} \quad r_{i,t}(\theta) > 1 + \epsilon.$$

Then the clipped PPO objective becomes

$$L_{i,t}^{\text{PPO}}(\theta) = (1 + \epsilon) \hat{A}_{i,t}.$$

This term is constant with respect to θ , and therefore

$$\nabla_{\theta} L_{i,t}^{\text{PPO}}(\theta) = 0.$$

Thus, when the probability of a positive-advantage token has already increased beyond the clipping range, PPO removes the gradient contribution from this token.

Similarly, if

$$\hat{A}_{i,t} < 0 \quad \text{and} \quad r_{i,t}(\theta) < 1 - \epsilon,$$

then the clipped PPO objective becomes

$$L_{i,t}^{\text{PPO}}(\theta) = (1 - \epsilon) \hat{A}_{i,t},$$

which is also constant with respect to θ . Hence,

$$\nabla_{\theta} L_{i,t}^{\text{PPO}}(\theta) = 0.$$

In this case, PPO removes the gradient contribution when the probability of a negative-advantage token has already decreased beyond the clipping range.

By contrast, CISPO uses the clipped importance sampling ratio as a stop-gradient coefficient:

$$\mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}) \right].$$

Since

$$\text{sg}(\hat{r}_{i,t}(\theta))$$

is treated as a constant during backpropagation, the gradient of the CISPO objective is

$$\nabla_{\theta} \mathcal{J}_{\text{CISPO}}(\theta) = \mathbb{E} \left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \text{sg}(\hat{r}_{i,t}(\theta)) \hat{A}_{i,t} \nabla_{\theta} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}) \right].$$

Therefore, even when the ratio $r_{i,t}(\theta)$ falls outside the clipping interval, CISPO does not make the gradient of that token vanish. Instead, the clipped ratio $\hat{r}_{i,t}(\theta)$ controls the magnitude of the gradient, while the token still contributes a policy-gradient signal through

$$\nabla_{\theta} \log \pi_{\theta}(o_{i,t} \mid q, o_{i,<t}).$$

In short, PPO clips the surrogate objective, which can cause clipped tokens to have zero gradient. CISPO clips the importance sampling ratio and uses it as a stop-gradient weight, so clipped tokens can still contribute to the policy gradient.

Chapter 3

Agentic RL

3.1 Context Compression

3.1.1 Motivation

In a long-horizon agent rollout, the working context contains the original instructions, model-generated actions, and environment observations. If every turn is appended, the context eventually reaches the model’s context limit. Even before that point, a very long context increases rollout cost and can degrade instruction following. Merely applying a summarizer at inference time does not address the training problem: after old interactions are removed, the summary determines which information is available to every subsequent action. Summary generation is therefore part of the policy, rather than a lossless preprocessing operation.

3.1.2 Compaction Mechanism

Both SUPO (Lu et al., 2025) and CompactionRL (Y. Li et al., 2026) make this idea trainable. Let the interaction history after turn t be

$$h_t = (p_0, z_1, \dots, z_t), \quad z_i = (\mathbf{a}_i, o_i), \quad (3.1)$$

where p_0 is the fixed initial context, $\mathbf{a}_i$ is a policy-generated assistant action span, and o_i is the corresponding environment observation. When a deterministic length threshold is reached, a fixed summarization instruction q_{sum} is appended and the same trainable policy generates

$$S_t \sim \pi_\theta(\cdot \mid h_t \oplus q_{\text{sum}}). \quad (3.2)$$

The rollout then resumes from a reconstructed context of the generic form

$$\bar{h}_t = p_{\text{keep}} \oplus u_{\text{resume}}(S_t) \oplus \text{Tail}_\kappa(z_1, \dots, z_t). \quad (3.3)$$

Here $\text{Tail}_\kappa(z_1, \dots, z_t) = (z_{t-\kappa+1}, \dots, z_t)$ denotes the κ most recent complete action–observation pairs. SUPO uses $\kappa = 0$, while CompactionRL uses $\kappa = 2$ by default and reduces it when necessary to fit the context budget. The retained prefix p_{keep} is also implementation-specific. These details do not change the shared causal mechanism: the generated summary replaces most of the old history and affects the terminal task reward through all future decisions.

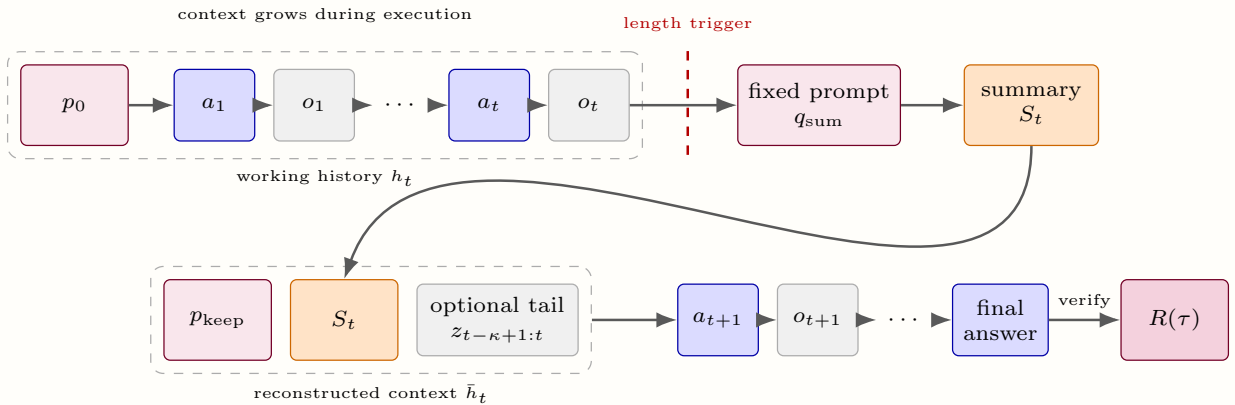


Figure 3.1: Shared trainable compaction mechanism in SUPO and CompactionRL. A deterministic length rule triggers a fixed summary instruction, and the same policy that performs task actions generates the summary. The working context is reconstructed from retained initial context, the summary, and an optional recent-history tail before the same rollout continues. The final task reward trains both execution and summary tokens. The exact retained prefix and history tail are method-specific.

It is useful to make precise what is learned. In both methods, the *time of compaction is forced by a length rule*; the policy does not learn whether or when to compact. The policy learns the summary tokens and the ordinary

execution tokens. Partition a compacted rollout into context-bounded multi-turn segments $\tau = (\sigma_1, \dots, \sigma_K)$. A typical nonfinal segment has the form

$$\sigma_k = (p_{\text{keep}}, u_{\text{resume}}(S_{k-1}), \text{Tail}_{\kappa_k}, (\mathbf{a}_{k,1}, o_{k,1}), \dots, (\mathbf{a}_{k,m_k}, o_{k,m_k}), q_{\text{sum}}, S_k), \quad (3.4)$$

where $\mathbf{a}_{k,t}$ is an assistant action span and $o_{k,t}$ is its environment observation. The previous summary is absent from σ_1 ; the final segment omits q_{sum}, S_K and instead ends with the final answer.

Each assistant action span is itself a token sequence; for example, $\mathbf{a}_{k,t} = (y_{k,t,1}, \dots, y_{k,t,L_{k,t}})$, and the summary S_k also contains many policy-generated tokens. Flatten the generated tokens in the execution and summary spans of σ_k in chronological order as $y_{k,1}, \dots, y_{k,n_k}$. Let $x_{k,i}$ be the complete visible prefix immediately before token $y_{k,i}$ is sampled. Thus observations are incorporated into later prefixes $x_{k,i}$ but are not themselves optimized tokens. Let $\mathcal{M}(\tau)$ collect all such policy-token coordinates (k, i) . Figure 3.2 visualizes this multi-turn structure.

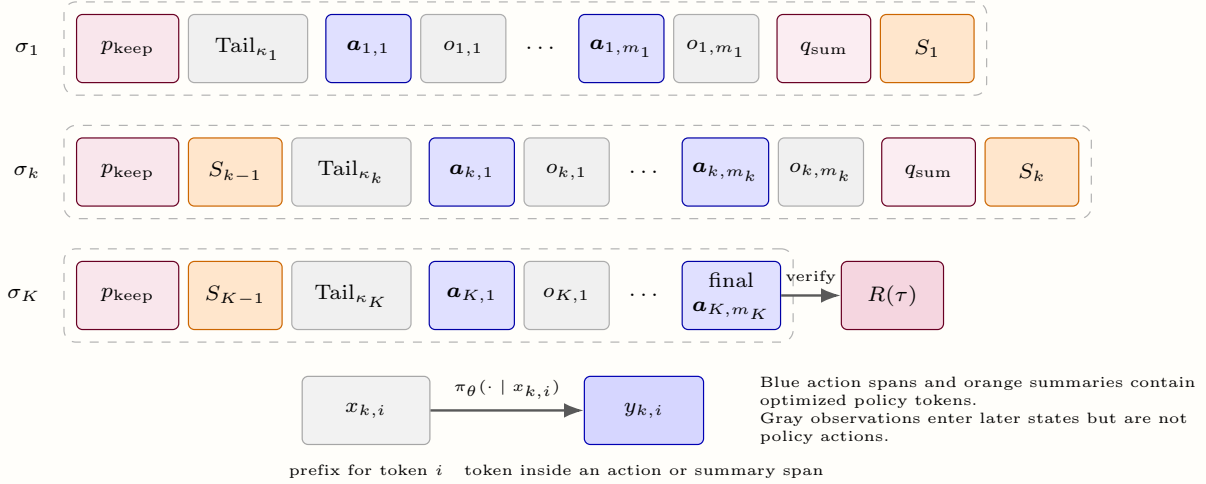


Figure 3.2: Multi-turn segment structure and policy-token indexing. Every segment begins with retained context, the preceding summary when available, and an optional recent tail, followed by assistant action–observation pairs. A nonfinal segment ends by generating its next summary. After flattening only the policy-generated execution and summary tokens, (k, i) selects token $y_{k,i}$ and $x_{k,i}$ is its pre-token prefix. Observation tokens affect later prefixes but do not belong to $\mathcal{M}(\tau)$.

With this indexing, the score-function identity gives

$$\nabla_{\theta} \mathcal{J}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[R(\tau) \sum_{(k,i) \in \mathcal{M}(\tau)} \nabla_{\theta} \log \pi_{\theta}(y_{k,i} \mid x_{k,i}) \right]. \quad (3.5)$$

Thus a separate summary-quality reward is not required: a summary is reinforced when the information it preserves helps the later policy obtain a larger task reward. The main difference between the two algorithms is how the common terminal reward is converted into an advantage for each token.

3.1.3 CompactionRL: PPO with Cross-Segment GAE

CompactionRL uses a fixed context budget C and triggers compaction when

$$C - |h_t| < T_{\text{comp}}. \quad (3.6)$$

It treats each $(\mathbf{a}_i, o_i)$ pair as atomic, so a tool call is not separated from its observation. After sampling S_t , the new context contains the system prompt, a fixed resume template carrying S_t , and the κ most recent action–observation pairs. The default is $\kappa = 2$, reduced if needed to fit the context budget. We use the context-bounded segments $\sigma_1, \dots, \sigma_K$ defined in Equation (3.4); execution and summary tokens in every segment are optimized under the same rollout reward. Here $\mathcal{M}$ denotes all token coordinates (k, i) in these segments.

Token-normalized PPO objective. For generated token $y_{k,i}$ in σ_k , conditioned on its visible prefix $x_{k,i}$, define the ratio against the rollout policy by

$$\rho_{k,i}(\theta) := \frac{\pi_{\theta}(y_{k,i} \mid x_{k,i})}{\pi_{\theta_{\text{old}}}(y_{k,i} \mid x_{k,i})}. \quad (3.7)$$

CompactionRL maximizes

$$\mathcal{J}_{\text{CompactionRL}}(\theta) := \frac{1}{|\mathcal{M}|} \sum_{(k,i) \in \mathcal{M}} \min \left\{ \rho_{k,i}(\theta) \hat{A}_{k,i}, \text{clip}(\rho_{k,i}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{k,i} \right\}. \quad (3.8)$$

The single denominator counts all optimized tokens in the batch. This avoids giving every compacted segment equal weight regardless of its length, which would over-weight rollouts split into many short segments. It does not make all rollouts equally weighted: a rollout containing more optimized tokens can still contribute more terms.

Cross-segment generalized advantage estimation. Let segment σ_k contain n_k optimized tokens. CompactionRL first computes a local GAE

$$\delta_{k,i} := r_{k,i} + \gamma V_\phi(x_{k,i+1}) - V_\phi(x_{k,i}), \quad (3.9)$$

$$A_{k,i}^{\text{loc}} := \sum_{\ell=0}^{n_k-i} (\gamma\lambda)^\ell \delta_{k,i+\ell}. \quad (3.10)$$

If this calculation is performed independently on every segment, placing the shared terminal reward at every segment boundary would make that reward appear too close to tokens in early segments. Let

$$N_{>k} := \sum_{m>k} n_m \quad (3.11)$$

be the number of optimized tokens in later segments of the same rollout. CompactionRL corrects the local estimate by

$$\hat{A}_{k,i} := (\gamma\lambda)^{N_{>k}} A_{k,i}^{\text{loc}}. \quad (3.12)$$

In particular, the terminal-reward contribution at token (k,i) is discounted by $(\gamma\lambda)^{N_{>k}+n_k-i}$, matching its token distance to the final outcome in the concatenated rollout. This exact-distance statement concerns the terminal-reward term: Equation (3.12) rescales the entire local advantage, including its value residuals, so it is an approximation rather than ordinary GAE over one uninterrupted state sequence. The paper also does not fully specify the token-level placement of $r_{k,i}$ or whether the value-regression target uses the local or corrected advantage.

A critic V_ϕ is trained with the standard PPO value-regression loss, so the method can estimate token-dependent advantages even with one rollout per prompt. In the reported implementation, the critic is pretrained and receives two updates per policy update. It uses

$$\lambda = 1 - \frac{1}{\alpha l}, \quad \alpha = 1.5, \quad (3.13)$$

where the paper calls l the response length but does not specify its exact interpretation across multi-response execution segments.

3.1.4 SUPO: GRPO with Rollout-Level Group Advantages

SUPO augments a multi-turn tool-use MDP with a length threshold L . Once the threshold is crossed, the policy is prompted to summarize the current subtrajectory. The next subtrajectory starts from the original prompt plus that summary; unlike CompactionRL, no recent action–observation tail is kept. The practical rollout algorithm also discards the action–observation pair that caused the threshold crossing before requesting the summary. This keeps the summary itself within the training context limit; the paper’s theoretical transition, in contrast, retains that pair. A complete rollout τ^j with I^j summaries is consequently stored as $I^j + 1$ subtrajectories.

Advantage estimation before splitting. For each prompt, SUPO samples G complete compacted rollouts from $\pi_{\theta_{\text{old}}}$ and obtains terminal rewards $\{R^1, \dots, R^G\}$. Its GRPO-style advantage is computed over these original rollouts:

$$\hat{A}^j := \frac{R^j - \text{mean}(\{R^{j'}\}_{j'=1}^G)}{\text{std}(\{R^{j'}\}_{j'=1}^G)}. \quad (3.14)$$

The same scalar $\hat{A}^j$ is then assigned to every execution and summary token in all $I^j + 1$ subtrajectories originating from rollout j . The important detail is that R^j appears only once in the group statistics. Computing the mean and standard deviation after splitting would repeat R^j $I^j + 1$ times and bias the baseline toward rollouts that compact more often. As with ordinary GRPO, the implementation must handle a group whose rewards have zero standard deviation, but the paper does not state the safeguard used.

SUPO objective and overlong mask. Write response span $\mathbf{a}_t^j$ as $\mathbf{a}_t^j = (y_{t,1}^j, \dots, y_{t,L_t^j}^j)$, and let $x_{t,\ell}^j$ be the visible prefix before token $y_{t,\ell}^j$. The token-level ratio is

$$\rho_{t,\ell}^j(\theta) := \frac{\pi_\theta(y_{t,\ell}^j | x_{t,\ell}^j)}{\pi_{\theta_{\text{old}}}(y_{t,\ell}^j | x_{t,\ell}^j)}. \quad (3.15)$$

Let $\mathcal{M}^j$ contain all generated-token positions in all subtrajectories of rollout j , and let

$$m^j := \mathbb{I}\{\tau^j \text{ emits a final answer before truncation by } H \text{ or } S\}. \quad (3.16)$$

Here H and S are the maximum numbers of interaction steps and summaries. Crucially, m^j is a *completion mask*, not a success mask: a completed but incorrect answer still has $m^j = 1$ and therefore remains a negative training example. An overlong rollout instead ends because an external compute limit is reached, without producing a final answer. Its failure reward does not reveal which earlier decision caused nontermination. Because SUPO broadcasts the same $\hat{A}^j$ to every execution and summary token, retaining a negative objective term for such a rollout would suppress all of those decisions together, including summaries that may have supported useful continued search. This creates a shortcut incentive to avoid summarization and force an early answer.

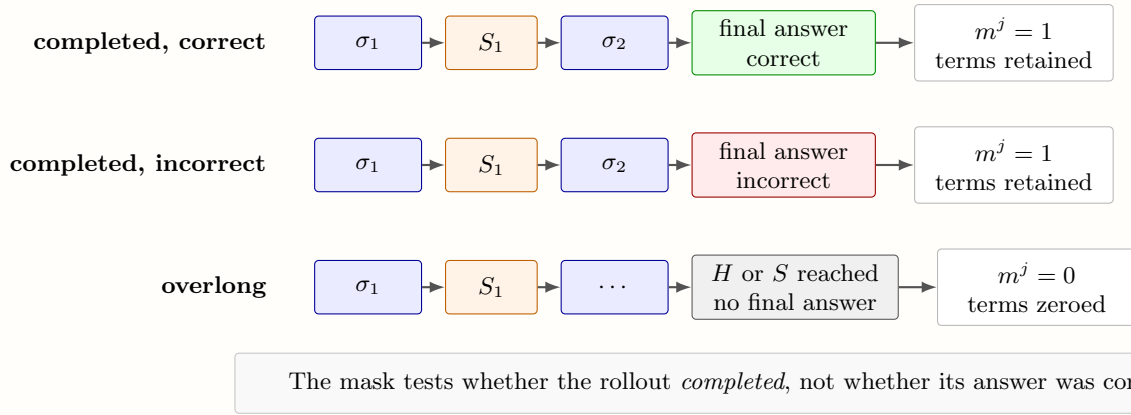


Figure 3.3: Effect of SUPO’s overlong mask. Both correct and incorrect completed rollouts contribute policy-gradient terms. Only a rollout truncated by the step limit H or summary limit S before emitting a final answer has all of its token-level terms zeroed.

SUPO then maximizes the globally token-normalized objective

$$\mathcal{J}_{\text{SUPO}}(\theta) := \frac{1}{\sum_{j=1}^G |\mathcal{M}^j|} \sum_{j=1}^G \sum_{(t,\ell) \in \mathcal{M}^j} m^j \min \left\{ \rho_{t,\ell}^j(\theta) \hat{A}^j, \text{clip} \left(\rho_{t,\ell}^j(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) \hat{A}^j \right\}. \quad (3.17)$$

The mask removes the overlong rollout’s own policy-gradient terms. As written, its reward can still affect the detached group mean and standard deviation in Equation (3.14). In the paper’s ablation, removing the mask causes summarization usage to collapse and reduces conditional success. The reported experiments use $G = 8$, asymmetric clipping $\epsilon_{\text{low}} = 0.20$ and $\epsilon_{\text{high}} = 0.28$, and no KL or entropy penalty.

Comparison. The apparent tension between the papers is resolved by the unit at which the advantage is computed. CompactionRL avoids group-relative estimation and uses a critic to give each token a temporally corrected advantage; it therefore supports group size one. SUPO remains critic-free by computing one relative advantage per *complete rollout before segmentation* and broadcasting it to all of that rollout’s tokens. SUPO thereby handles the variable number of subtrajectories without duplicating rewards in the group baseline, but it cannot distinguish an early summary token from a late execution token within the same rollout. The compaction mechanism is shared; the principal algorithmic difference is fine-grained temporal credit assignment versus a coarser, critic-free group-relative baseline.

3.2 Asynchronous Algorithms

3.2.1 Introduction

For agentic workloads, especially long-horizon tasks, rollout lengths are highly variable, so short trajectories finish quickly while long ones become stragglers.

Asynchronous RL algorithms are proposed to mitigate this imbalanced generation overhead by consuming roll-outs continuously as they arrive, improving utilization and wall-clock efficiency. However, asynchrony introduces one challenge, where each trajectory can be generated by multiple versions of the old rollout model, which leads to more unpredictable and severe off-policy, and thus harms the training stability. Recall that in the Standard PPO, the objective is defined as

$$\max_{\theta} \mathcal{J}_{\text{PPO-clip}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_{t=0}^{T-1} \min \left\{ \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_{\phi}(s_t, a_t), \text{clip} \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{\phi}(s_t, a_t) \right\} \right],$$

controls policy update size via the ratio $\frac{\pi_{\theta}}{\pi_{\theta_{\text{old}}}}$, where $\pi_{\theta_{\text{old}}}$ is assumed to be both the *proximal policy* (the anchor policy for PPO clipping) and the *behavior policy* (the policy that collected the data); both terms are defined in (Hilton et al., 2022), which was primarily proposed to achieve batch size-invariance for PPO.

The key idea is to decouple these two roles. Let $\pi_{\theta_{\text{prox}}}$ denote the proximal policy that anchors the clipping region, and let $\pi_{\theta_{\text{beh}}}$ denote the behavior policy that collected the trajectories. Then a three-policy version of PPO-Clip can be written as

$$\max_{\theta} \mathcal{J}_{\text{PPO-Decoupled}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{beh}}}} \left[\sum_{t=0}^{T-1} w_t \min \left\{ r_t(\theta) \hat{A}_{\phi}(s_t, a_t), \text{clip} (r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_{\phi}(s_t, a_t) \right\} \right],$$

where

$$w_t := \frac{\pi_{\theta_{\text{prox}}}(a_t|s_t)}{\pi_{\theta_{\text{beh}}}(a_t|s_t)}, \quad r_t(\theta) := \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{prox}}}(a_t|s_t)}.$$

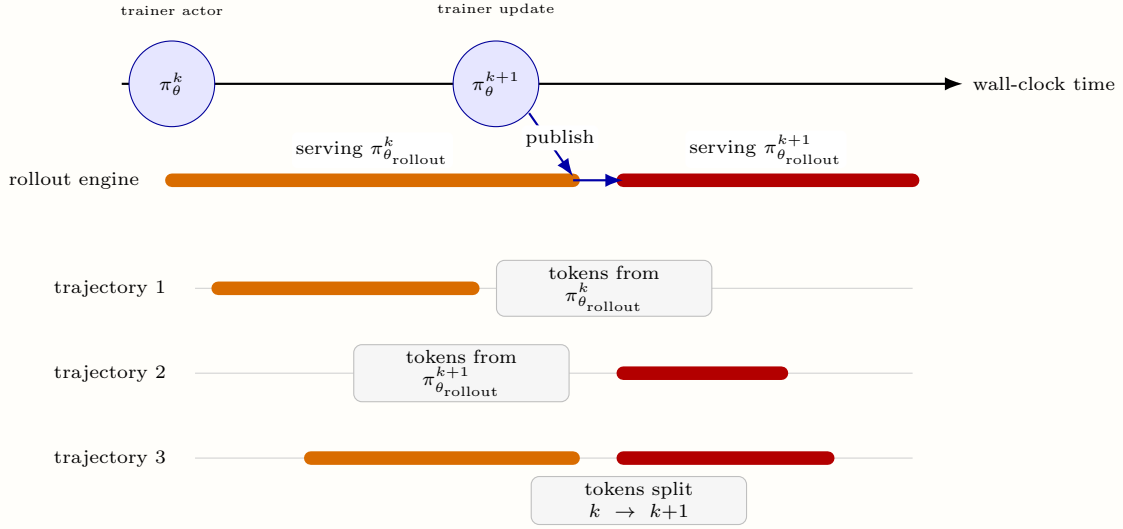
Here, w_t is the importance sampling correction from the behavior policy to the proximal policy. During one policy update, $\pi_{\theta_{\text{prox}}}$ and $\pi_{\theta_{\text{beh}}}$ are fixed, so w_t is treated as a constant with respect to θ . In contrast, $r_t(\theta)$ is the PPO ratio used to control how far the current policy π_{θ} moves away from the proximal policy $\pi_{\theta_{\text{prox}}}$.

Remark.

- If $\pi_{\theta_{\text{beh}}} = \pi_{\theta_{\text{prox}}} = \pi_{\theta_{\text{old}}}$, then $w_t = 1$ and the objective reduces to the standard PPO-Clip objective.
- The behavior policy $\pi_{\theta_{\text{beh}}}$ can be stale or generated by a different worker, while the update size is still controlled relative to $\pi_{\theta_{\text{prox}}}$.
- The clipping mechanism is preserved because the clipped ratio is $r_t(\theta) = \pi_{\theta}(a_t|s_t)/\pi_{\theta_{\text{prox}}}(a_t|s_t)$, not the behavior correction w_t .

For the asynchronous discussion, it is clearer to write the proximal policy as $\pi_{\theta_{\text{old}}}$ and the behavior policy as $\pi_{\theta_{\text{rollout}}}$. The policy $\pi_{\theta_{\text{old}}}$ is the frozen PPO anchor used in the clipping ratio, while $\pi_{\theta_{\text{rollout}}}$ is the model that actually generated the sampled token or action. For more details, see the [veRL²](#) discussion and the [Slime²](#) reference implementation.

Consequently, the data consumed by a trainer update may not have a single behavior policy. Some trajectories may have been generated before the rollout engine refreshed, while later trajectories may use the refreshed rollout weights; in a long rollout, even different tokens in the same trajectory can correspond to different rollout snapshots. See [Figure 3.4](#) for a visualization.



A training window can contain rollouts from different rollout-engine snapshots.

The stored rollout log-probability identifies the behavior policy for each sampled token.

Figure 3.4: Policy lag comes from the time gap between the trainer actor update and the rollout engine refresh. During this gap, the trainer has advanced to π_{θ}^{k+1} while the rollout engine may still generate tokens with $\pi_{\theta_{\text{rollout}}}^k$; after refresh, later tokens are generated with $\pi_{\theta_{\text{rollout}}}^{k+1}$. A trajectory that spans the refresh can therefore contain tokens from both rollout snapshots.

Thus, exact off-policy correction requires the behavior probability for the particular sampled token/action, namely the rollout-time value $\pi_{\theta_{\text{rollout}}}(a_t | s_t)$ used when that token/action was sampled. In practice, this means storing the rollout log-probability together with each generated token. If these log-probabilities are not stored, then one would need to keep enough rollout-policy checkpoints to recompute them later. The required history is therefore a history of behavior probabilities, or equivalently rollout-policy snapshots. In the long horizon setting, maintaining the history is not feasible in practice.

In this part, we discuss various asynchronous RL algorithms target to address various issues arised in the asynchronous RL.

3.2.2 SAO: Single-Rollout Asynchronous Optimization for Agentic Reinforcement Learning

3.2.2.1 Algorithm

To address the policy lag issue, SAO (Hou et al., 2026) proposes several modifications over the PPO-Clip objective to reach the following objective:

$$\max_{\theta} \mathcal{J}_{\text{SAO}}(\theta) := \mathbb{E}_{\tau \sim \pi_{\theta_{\text{rollout}}}} \left\{ \sum_{t=0}^{T-1} \left[\text{clip} \left(\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{rollout}}}(a_t | s_t)}, 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}} \right) \hat{A}_{\phi}(s_t, a_t) \right] \log \pi_{\theta}(a_t | s_t) \right\}, \quad (3.18)$$

where (3.18) is written in REINFORCE form (1.7): the clipped importance ratio times the advantage is treated as a detached weight, and the explicit $\log \pi_{\theta}(a_t | s_t)$ term provides the policy-gradient direction.

Remark

1. SAO drops the proximal policy $\pi_{\theta_{\text{old}}}$ and uses the behavior policy $\pi_{\theta_{\text{rollout}}}$ for importance sampling:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{rollout}}}(a_t | s_t)}.$$

This also avoids recomputing probabilities under $\pi_{\theta_{\text{old}}}$ by reusing the log-probabilities generated during rollout.

2. SAO applies a different trust region by clipping the rollout ratio to $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$. As shown in Figure 3.5, PPO clips by asking “Has this sample already moved too far in the direction that appears beneficial?”, while SAO clips by asking “Is the current policy still sufficiently close to the policy that generated this token for this sample to be trustworthy?”.

- **PPO’s asymmetric construction is useful** when the advantage estimation is reliable, the data is fresh, the large ratio arose from parameter coupling or prior minibatch updates, and we want the sample to help undo an undesirable policy shift. For example, if $A = -1$ and $r = 10$, PPO-Clip keeps the directionally corrective term $rA = -10$, which pushes the sampled action’s probability down, while SAO clips it. Similarly, if $A = 1$ and $r = 0.1$, PPO keeps the corrective gradient, while SAO clips it. Because only one side is saturated for each advantage sign, PPO generally uses more of the batch and supports repeated minibatch optimization over collected data. However, PPO’s retained corrective gradients can become dangerous in asynchronous or heavily off-policy training.
- **SAO’s symmetric construction is helpful** in asynchronous or heavily off-policy training. For $A < 0$ and $r \gg 1$, the factor rA can dominate a minibatch. That may be acceptable if A is accurate and the sample is still relevant, but it can be destructive when the sample was generated by a substantially older actor, the trajectory context is highly off-policy, or one extreme token dominates the aggregate gradient. SAO bounds the importance ratio regardless of the advantage sign, giving direct control over this source of gradient heavy tails.

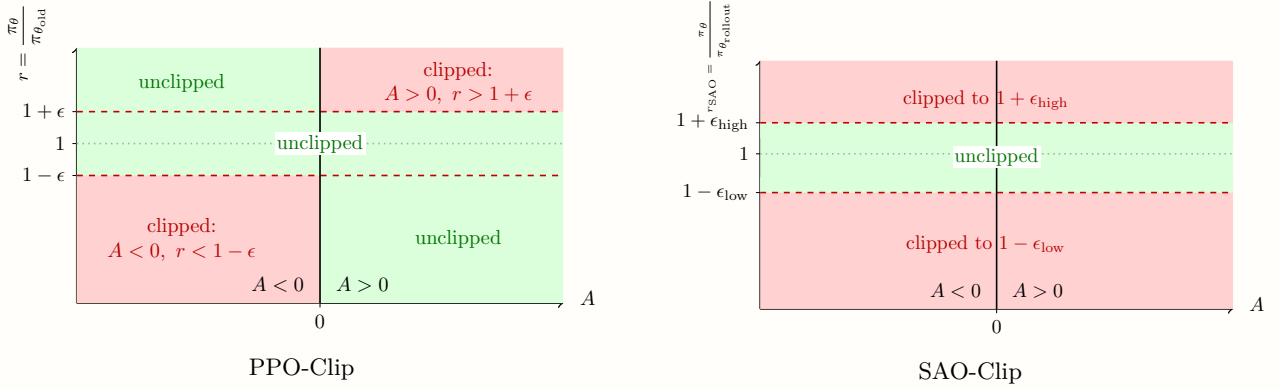


Figure 3.5: PPO-Clip and SAO-Clip trust regions in the (A, r) plane. PPO clips only in the regions selected by the clipped surrogate objective, while SAO directly clips the rollout-ratio $r = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{rollout}}}(a_t|s_t)}$ to $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$.

3. The strict two-sided gate has costs. First, once a token is outside the band, it gives no direct gradient, even when its advantage would have produced a restoring update. This is a distinction between filtering and constraining: SAO filters data; it does not project or roll the policy back into a feasible region. Second, importance sampling is unbiased only without truncation under the usual assumptions. PPO introduces bias through one-sided clipping, while SAO introduces more bias by dropping both tails. In particular, dropping a negative-advantage outlier can make the observed surrogate look artificially favorable because a large negative contribution has disappeared. Therefore, SAO’s gated objective does not have PPO’s uniform pointwise pessimism property. Third, clipping reduces the effective batch size: if many tokens leave the band, the nominal batch size may remain large while the number of contributing tokens collapses.
4. The paper does not specify every detail of the advantage estimator $\hat{A}_\phi$. We sketch a consistent token-level interpretation below.

3.2.2.2 Guessed Advantage Estimator

Trajectory, action spans, and states. Let P denote the fixed task context supplied before interaction begins. It may contain system instructions, user instructions, task metadata, and chat-template tokens preceding the environment state. Before the agent takes its first action, the environment provides an initial observation o_0 . The interaction then proceeds as

$$o_0 \xrightarrow{a_0, R_0} o_1 \xrightarrow{a_1, R_1} o_2 \dots \xrightarrow{a_{M-1}, R_{M-1}} o_M.$$

Equivalently, the chronological trace is

$$P, o_0, a_0, o_1, a_1, \dots, o_{M-1}, a_{M-1}, o_M.$$

Here o_i is the observation available before action a_i ; a_i is the i -th model-generated action span; after executing a_i , the environment returns reward R_i and next observation o_{i+1} ; and o_M is the final observation after the last sampled action. Action i consists of n_i policy-generated tokens:

$$a_i = (a_{i,0}, a_{i,1}, \dots, a_{i,n_i-1}).$$

These are exactly the tokens sampled from the policy during the i -th assistant generation. Tokens inserted by the chat template, such as an assistant-role marker, are not included in a_i unless they are actually sampled by the policy. Similarly, observation tokens are not part of a_i because they are supplied by the environment rather than generated by the policy.

Let σ_i denote the serialized assistant prefix placed immediately before generation of action a_i . Depending on the chat template, this may include tokens such as `<assistant>`, or tool-call formatting tokens. Define c_i as the complete token sequence available immediately before the first generated token $a_{i,0}$. Concretely,

$$c_i := \text{Serialize}(P, o_0, a_0, o_1, a_1, \dots, a_{i-1}, o_i, \sigma_i) \text{ for } i \geq 0.$$

For the j -th token of action span i , define $a_{i,<j} := (a_{i,0}, \dots, a_{i,j-1})$. The policy-decision state before generating token $a_{i,j}$ is $s_{i,j} := c_i \| a_{i,<j}$, where $\|$ denotes token-sequence concatenation. Thus $s_{i,0} = c_i$. The token-generation rule is

$$a_{i,j} \sim \pi_\theta(\cdot \mid s_{i,j}),$$

and the value associated with the same pre-token state is

$$v_{i,j} := V_\phi(s_{i,j}).$$

This notation keeps the actor and critic aligned: both **evaluate the prefix before the token is sampled**.

Successor states. Inside action span i , for $j < n_i - 1$, the next decision state is

$$s_{i,j+1} = s_{i,j} \| a_{i,j}.$$

At the end of the action span, the agent emits a_{i,n_i-1} , the environment executes the completed action, and returns o_{i+1} . The next policy-decision state is

$$s_{i+1,0} = c_{i+1}.$$

Expanded,

$$s_{i+1,0} = \text{Serialize}(P, o_0, a_0, o_1, \dots, o_i, a_i, o_{i+1}, \sigma_{i+1}).$$

The complete action a_i , not merely its final token, determines the environment transition. The final token is simply the token-level decision to which the action-level reward R_i is conventionally aligned. Observation o_{i+1} is part of the next decision state, but individual observation tokens are not separate policy-decision steps.

For two action spans, the causal structure is

$$P, o_0, \sigma_0 \xrightarrow{\text{generate } a_0} a_0 \xrightarrow{\text{environment}} R_0, o_1, \sigma_1 \xrightarrow{\text{generate } a_1} a_1 \xrightarrow{\text{environment}} R_1, o_2.$$

The corresponding policy-decision sequence is

$$s_{0,0}, s_{0,1}, \dots, s_{0,n_0-1}, s_{1,0}, s_{1,1}, \dots,$$

where

$$s_{0,0} = \text{Serialize}(P, o_0, \sigma_0), \quad s_{1,0} = \text{Serialize}(P, o_0, a_0, o_1, \sigma_1).$$

This is the sequence on which skip-observation GAE should operate. The observation o_1 is part of $s_{1,0}$, but its individual serialized tokens are not separate policy-decision steps.

Transformer and value-head alignment. Let

$$x_0, x_1, \dots, x_{T-1}$$

be the complete serialized token sequence, including prompt tokens, observations, action tokens, role tokens, tool delimiters, and assistant-prefix tokens. Let $p_{i,j}$ be the physical position of action token $a_{i,j}$, so

$$x_{p_{i,j}} = a_{i,j}.$$

By the state definition,

$$s_{i,j} = x_{<p_{i,j}}.$$

Write the value model as a causal transformer body followed by a scalar head:

$$h_t^V = F_\phi^{\text{body}}(x_{\leq t}), \quad \bar{v}_t = g_\phi^{\text{head}}(h_t^V),$$

with $\phi = (\phi_{\text{body}}, \phi_{\text{head}})$. The raw output $\bar{v}_t$ evaluates the prefix through physical position t :

$$\bar{v}_t = V_\phi(x_{\leq t}).$$

Since the state before action token $a_{i,j}$ is

$$s_{i,j} = x_{<p_{i,j}} = x_{\leq p_{i,j}-1},$$

its value is

$$v_{i,j} = V_\phi(s_{i,j}) = \bar{v}_{p_{i,j}-1}.$$

This means:

- before an interior action token, the relevant hidden state is usually located at the preceding action token;
- before the first token of a new action span, the relevant hidden state is located at the final serialized token before generation begins;
- that final preceding token may be an observation token, tool delimiter, or assistant-role token.

For the first token after observation o_{i+1} ,

$$v_{i+1,0} = V_\phi(s_{i+1,0}) = \bar{v}_{p_{i+1,0}-1}.$$

Because $s_{i+1,0} = x_{<p_{i+1,0}}$ contains o_{i+1} , the critic prediction $v_{i+1,0}$ is conditioned on the complete next observation.

Reward alignment. Let $r_{i,j}$ be the reward assigned to the transition caused by choosing token $a_{i,j}$. Inside an action span, the usual assignment is

$$r_{i,j} = 0, \quad j < n_i - 1,$$

unless token-level shaping rewards are used. If the environment returns reward R_i after executing the complete action span a_i , assign it to the span's final token:

$$r_{i,n_i-1} = R_i.$$

Equivalently,

$$r_{i,j} = \begin{cases} 0, & j < n_i - 1, \\ R_i, & j = n_i - 1. \end{cases}$$

For a trajectory with only one terminal verifier reward R , all token rewards are zero except the final token of the final action span. This does not claim that the final token alone caused the reward; earlier action tokens receive credit through the backward GAE recursion.

Token-level TD residuals and advantages. For an interior token $j < n_i - 1$, the next policy-decision state is $s_{i,j+1}$. Therefore,

$$\delta_{i,j} := r_{i,j} + \gamma v_{i,j+1} - v_{i,j},$$

and the GAE recursion is

$$A_{i,j}^{(\lambda)} := \delta_{i,j} + \gamma \lambda A_{i,j+1}^{(\lambda)}.$$

Normally $r_{i,j} = 0$ for interior tokens, so $\delta_{i,j} = \gamma v_{i,j+1} - v_{i,j}$.

For the final token of a nonterminal action span, the next decision is $a_{i+1,0}$ after observation o_{i+1} has been incorporated into the context. Thus,

$$\delta_{i,n_i-1} := R_i + \gamma v_{i+1,0} - v_{i,n_i-1}, \text{ and } A_{i,n_i-1}^{(\lambda)} := \delta_{i,n_i-1} + \gamma \lambda A_{i+1,0}^{(\lambda)}.$$

This is the well-defined version of the Hou et al. (2026) (4) and (5). The observation tokens are included in the next state, but no separate GAE steps are inserted for each observation token.

- Final token of a terminal episode. Suppose action span $M - 1$ terminates the episode. There is no next policy-decision state, so set $v_{\text{terminal}} = 0$ and $A_{\text{terminal}} = 0$. Then

$$\delta_{M-1,n_{M-1}-1} := R_{M-1} - v_{M-1,n_{M-1}-1},$$

and

$$A_{M-1,n_{M-1}-1}^{(\lambda)} := \delta_{M-1,n_{M-1}-1}.$$

This is the initialization point for the backward recursion in a completed episode.

- Truncated but nonterminal trajectory. A time limit, maximum-token limit, or context limit is not necessarily a true terminal state. Let s_{boot} be the valid policy-decision state from which the rollout would have continued, and define

$$v_{\text{boot}} := V_{\phi}(s_{\text{boot}}).$$

For the last sampled action token,

$$\delta_{\text{last}} := r_{\text{last}} + \gamma v_{\text{boot}} - v_{\text{last}}.$$

Since no later TD residuals were sampled, initialize the GAE carry as $A_{\text{next}} := 0$, giving

$$A_{\text{last}} := \delta_{\text{last}}.$$

The bootstrap value represents the unobserved future return. Setting $v_{\text{boot}} := 0$ for an ordinary truncation would incorrectly treat the truncation as failure or termination.

Actor advantage and critic target. SAO uses separate trace-decay choices for the actor and critic. Let

$$\lambda_{\text{policy}} := 1 - \alpha l^{-1}, \quad \lambda_{\text{critic}} := 1,$$

where l is the rollout length used by the estimator. Using the same TD residuals, compute two backward recursions:

$$A_{i,j}^{\pi} := \delta_{i,j} + \gamma \lambda_{\text{policy}} A_{\text{next}(i,j)}^{\pi},$$

and

$$A_{i,j}^V := \delta_{i,j} + \gamma \lambda_{\text{critic}} A_{\text{next}(i,j)}^V.$$

The actor uses $A_{i,j}^{\pi}$ in the policy objective. The critic target is

$$G_{i,j} := \text{stopgrad}(v_{i,j} + A_{i,j}^V),$$

and a typical value loss is

$$L_V = \frac{1}{2|\mathcal{A}|} \sum_{(i,j) \in \mathcal{A}} (V_{\phi}(s_{i,j}) - G_{i,j})^2,$$

where $\mathcal{A}$ is the set of model-generated action-token positions. Thus the value loss is defined only for states preceding model-generated action tokens. In the skip-observation formulation, the natural length is

$$l = \sum_i n_i,$$

the number of model-generated action tokens. Counting observation tokens would make tool-output verbosity change λ_{policy} , even though observation-token positions are excluded from the GAE recursion.

What “skip observation” means. Skipping observation tokens does not mean that the critic ignores observations. The critic evaluates

$$v_{i+1,0} = V_{\phi}(s_{i+1,0}),$$

and $s_{i+1,0}$ contains the observation returned after action a_i . Therefore, the observation directly affects both the next value prediction and the next policy distribution. What is skipped is only the insertion of individual observation-token positions into the GAE time axis:

- observation tokens are not policy actions;
- no policy advantages are assigned to observation tokens;
- no TD residuals are formed between consecutive observation-token positions;
- no GAE carry is repeatedly decayed through the observation’s token length;
- the last action token connects directly to the next generated action token.

Thus, the observation remains in the state, but observation-token positions are removed from the GAE recursion. The paper’s claim that this “filters out the stochasticity of environment feedback” should be interpreted carefully: the randomness and content of o_{i+1} still affect $s_{i+1,0}$, $v_{i+1,0}$, and $\pi_{\theta}(\cdot \mid s_{i+1,0})$. What the method removes is dependence on intermediate value predictions within the observation span, observation tokenization, observation length, and repeated $\gamma\lambda$ decay across externally generated tokens.

3.2.2.3 Practical Considerations

What to do?

- **Faster Value Update than Policy.** The authors attribute primary source of instability in single-rollback RL is the interdependence between the policy and the value function. If the value model V_ϕ is inaccurate, the advantage estimates $\hat{A}_\phi(s_t, a_t)$ become noisy, leading to destructive policy updates. Specifically, for every single gradient update applied to the policy π_θ , we enforce K updates to the value network V_ϕ (default $K = 2$).
- **Stabilizing Value Model Training via Parameter Freezing.** Instability of value model training, where the gradient norms of the value model are significantly larger than the corresponding policy model. Further decomposition shows that this instability originates primarily from the Full Attention layers. As a result, the authors freeze the parameters of the Full Attention layers during value model training.

What to monitor? A strong hybrid is often:

gross-staleness gate+PPO or soft rollback within the gate+explicit KL/TV monitoring.

For diagnosing whether the extra SAO masking is helping, the most informative measurements are not merely the aggregate clip fraction. Track the four A-by-r quadrants separately, the accepted-token fraction by actor version, importance-weight effective sample size, log-ratio quantiles, and the gradient norm contributed by the $A < 0, r > u$ quadrant. That last quadrant is where SAO is most likely to gain stability and where it sacrifices the strongest potentially corrective PPO signal.

Action Masking. Let the complete tokenized trajectory be $x_0, x_1, \dots, x_{T-1}$. Define the action mask as

$$M_p^{\text{action}} = \mathbf{1}\{x_p \text{ is generated by the policy}\}.$$

Suppose x_p is a generated action token, so $M_p^{\text{action}} = 1$. Its pre-action state is the prefix before that token,

$$s_p = x_{<p} = x_{\leq p-1}.$$

If the transformer value head emits

$$\bar{v}_t := V_\phi(x_{\leq t}),$$

then the value associated with action token x_p is

$$v_p := V_\phi(s_p) = V_\phi(x_{\leq p-1}) = \bar{v}_{p-1}.$$

Therefore, when the critic target for action token x_p is denoted by G_p , the physical-token value loss can be written as

$$L_V = \frac{1}{\sum_{p=1}^{T-1} M_p^{\text{action}}} \sum_{p=1}^{T-1} M_p^{\text{action}} (\bar{v}_{p-1} - \text{stopgrad}(G_p))^2.$$

Note that the index p starts at 1 because x_0 is not an action token (assuming non-empty context). Equivalently, reindex with $t = p - 1$:

$$L_V = \frac{1}{\sum_{t=0}^{T-2} M_{t+1}^{\text{action}}} \sum_{t=0}^{T-2} M_{t+1}^{\text{action}} (\bar{v}_t - \text{stopgrad}(G_{t+1}))^2.$$

Thus the implementation mask for critic outputs is a one-token-left shift of the action mask:

$$M_t^{\text{value}} = M_{t+1}^{\text{action}}, \quad t = 0, \dots, T-2.$$

Cleaner implementation convention. The least confusing implementation is to shift logits, values, labels, targets, and masks into the same coordinate system. After the shift, every index refers to the same predicted action position, so the explicit $t + 1$ offset disappears from the loss code:

```

1  # Raw output at position t represents/predicts the state or token at t+1.
2  pred_logits = logits[:, :-1]
3  pred_values = raw_values[:, :-1]
4
5  target_tokens = input_ids[:, 1:]
6  policy_advantages = advantages[:, 1:]
7  value_targets = returns[:, 1:]
8

```

```
9  decision_mask = action_mask[:, 1:]
10  dis_mask = ratio_in_trust_region[:, 1:] #  $1 - \epsilon_{low} < r < 1 + \epsilon_{high}$ 
11
12  policy_mask = decision_mask * dis_mask
13  value_mask = decision_mask
14
15  # The losses are conceptually
16  policy_loss = (
17      token_policy_loss(pred_logits, target_tokens, policy_advantages)
18      * policy_mask).sum() / policy_mask.sum().clamp_min(1)
19
20  value_loss = (
21      (pred_values - value_targets.detach()).square()
22      * value_mask).sum() / value_mask.sum().clamp_min(1)
```

Chapter 4

Reward Modeling

4.1 Reward Modeling

A reward model (RM) replaces a hand-written objective with a learned scoring function. For a prompt x and completion y , it estimates a scalar or a set of local scores that can be used for best-of- N selection, search, or policy optimization. Unlike an environment reward, this signal is inferred from human preferences or correctness annotations and is therefore only a proxy for the desired behavior.

This section uses x for a prompt, y for a completion, y_c for a chosen completion, and y_r for a rejected completion. It is adapted from Lambert’s reward-modeling chapter.¹

4.1.1 Bradley–Terry Preference Reward Model

4.1.1.1 Preference likelihood

The Bradley–Terry (BT) model assigns each item i a positive latent strength p_i . Its pairwise preference probability is

$$P(i \succ j) = \frac{p_i}{p_i + p_j}. \quad (4.1)$$

Writing $p_i = \exp(r_i)$ gives the logit parameterization

$$P(i \succ j) = \frac{\exp(r_i)}{\exp(r_i) + \exp(r_j)} = \sigma(r_i - r_j), \quad \sigma(z) = \frac{1}{1 + \exp(-z)}. \quad (4.2)$$

Consequently, only score differences are identifiable: replacing every score r_i by $r_i + C$ leaves all preference probabilities unchanged.

For an RM $r_\theta(x, y)$ and a preference triple $(x, y_c, y_r) \sim \mathcal{D}$,

$$P_\theta(y_c \succ y_r \mid x) = \sigma(r_\theta(x, y_c) - r_\theta(x, y_r)). \quad (4.3)$$

Maximum likelihood yields the final loss function:

$$\mathcal{L}_{\text{BT}}(\theta) = \mathbb{E}_{(x, y_c, y_r) \sim \mathcal{D}} -\log \sigma(r_\theta(x, y_c) - r_\theta(x, y_r)). \quad (4.4)$$

The log is applied before averaging; maximizing $\mathbb{E}[P_\theta(y_c \succ y_r \mid x)]$ is not the same likelihood objective.

4.1.1.2 Architecture

The standard architecture reuses a causal-transformer backbone and replaces its vocabulary projection with a scalar head. For prompt-completion tokens $z = (x, y)$, let $h_t \in \mathbb{R}^d$ be the final-layer hidden state. A common sequence score is

$$r_\theta(x, y) = w^\top h_{t_{\text{last}}} + b, \quad (4.5)$$

where t_{last} is the final non-padding token, normally an EOS token. Chosen and rejected sequences are scored independently by the same network. The loss in (4.4) couples only their scalar outputs.

```
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5 class BradleyTerryRM(nn.Module):
6     def __init__(self, backbone):
7         super().__init__()
8         # Transformer producing one d-dimensional hidden state per token.
9         self.backbone = backbone
10        # Shared linear head: final token representation -> scalar reward.
11        self.reward_head = nn.Linear(backbone.config.hidden_size, 1)
12
13    def forward(self, input_ids, attention_mask):
14        """
```

¹<https://rlhfbook.com/c/05-reward-models>

```

15     Score each complete prompt-response sequence.
16
17     input_ids:      token IDs, shape [batch_size, sequence_length]
18     attention_mask: 1 for real tokens and 0 for padding, same shape
19     returns:        one unnormalized reward logit per sequence, shape [B]
20     """
21     out = self.backbone(
22         input_ids=input_ids,
23         attention_mask=attention_mask,
24         output_hidden_states=True,
25         return_dict=True,
26     )
27
28     # Last transformer layer: one contextual representation per token.
29     hidden = out.hidden_states[-1] # [B, T, d]
30
31     # Locate the final non-padding token. Unlike mask.sum() - 1, this is
32     # correct for both left-padded and right-padded batches.
33     positions = torch.arange(
34         hidden.size(1), device=hidden.device
35     ).expand_as(attention_mask)
36     last = positions.masked_fill(attention_mask == 0, -1).max(dim=1).values
37     batch = torch.arange(hidden.size(0), device=hidden.device)
38     final_hidden = hidden[batch, last] # [B, d], usually EOS
39
40     # The score is not a probability; only score differences enter BT loss.
41     return self.reward_head(final_hidden).squeeze(-1) # [B]
42
43 def preference_loss(model, chosen, rejected):
44     """Negative log-likelihood that chosen is preferred to rejected."""
45     # Both dictionaries contain the same prompts but different completions.
46     # The same reward model (shared parameters) scores both sequences.
47     reward_chosen = model(**chosen)
48     reward_rejected = model(**rejected)
49
50     # -log sigmoid(r_chosen - r_rejected), averaged over the batch.
51     # Minimization pushes the chosen score above the rejected score.
52     return -F.logsigmoid(reward_chosen - reward_rejected).mean()

```

Listing 4.1: Minimal Bradley–Terry reward model.

Correct construction of the two inputs is critical: each contains the same prompt and exactly one completion, and the attention mask must select the actual final token under either left or right padding. Reward models are commonly trained for only one epoch because pairwise datasets are correlated and overfitting can rapidly degrade held-out ranking accuracy.

4.1.2 Outcome Reward Models

An outcome reward model (ORM) predicts whether a completion eventually reaches a correct final answer. It is trained from independently labeled examples (x, y, r) , where $r \in \{0, 1\}$ denotes final-answer correctness; paired chosen–rejected data is unnecessary.

One common ORM emits a binary logit $w_{\theta,t}$ at every completion token. The sequence outcome label is broadcast across completion positions while prompt and padding positions are masked:

$$\mathcal{L}_{\text{ORM}}(\theta) = -\mathbb{E}_{(x,y,r) \sim \mathcal{D}} \left[\frac{1}{\sum_t m_t} \sum_{t=1}^T m_t (r \log p_{\theta,t} + (1-r) \log(1-p_{\theta,t})) \right], \quad (4.6)$$

where $m_t = 1$ only for completion tokens and $p_{\theta,t} = \sigma(w_{\theta,t})$. These targets contain no direct information about whether an intermediate step is valid: every token receives the label determined by the final outcome.

A sequence-pooled alternative first averages logits and then applies the sigmoid,

$$\bar{p}_{\theta}(x, y) = \sigma \left(\frac{1}{\sum_t m_t} \sum_t m_t w_{\theta,t} \right), \quad (4.7)$$

followed by sequence-level binary cross entropy. In general, $\sigma(\text{mean}(w_t)) \neq \text{mean}(\sigma(w_t))$, so these aggregation orders are not interchangeable.

```

1  import torch.nn as nn
2  import torch.nn.functional as F
3
4  class OutcomeRM(nn.Module):
5      def __init__(self, backbone):
6          super().__init__()
7          self.backbone = backbone
8          self.outcome_head = nn.Linear(backbone.config.hidden_size, 1)
9
10     def forward(self, input_ids, attention_mask, labels=None):
11         out = self.backbone(
12             input_ids=input_ids,
13             attention_mask=attention_mask,
14             output_hidden_states=True,
15             return_dict=True,
16         )
17         hidden = out.hidden_states[-1]
18         logits = self.outcome_head(hidden).squeeze(-1) # [B, T]
19
20         if labels is None:
21             return {"logits": logits}
22
23         # labels: -100 for prompt/padding; 0 or 1 for completion tokens
24         mask = labels.ne(-100)
25         loss = F.binary_cross_entropy_with_logits(
26             logits[mask], labels[mask].float()
27         )
28         return {"loss": loss, "logits": logits}

```

Listing 4.2: Per-token outcome reward model.

At inference, the per-token probabilities can be reduced to a candidate score using the mean, the last few positions, the minimum, or a sum of log probabilities. The reduction must match validation-time behavior; products strongly penalize long completions and minimum aggregation is highly sensitive to one anomalous token.

4.1.3 Process Reward Models

A process reward model (PRM) evaluates intermediate reasoning steps rather than only the final answer. Let $s = (s_1, \dots, s_K)$ be an annotated reasoning trace and let $q_i \in \{0, 1\}$ indicate whether step s_i is valid conditioned on the prompt and previous steps. A binary PRM minimizes

$$\mathcal{L}_{\text{PRM}}(\theta) = -\mathbb{E}_{(x,s) \sim \mathcal{D}} \sum_{i=1}^K [q_i \log r_{\theta}(s_i | x, s_{<i}) + (1 - q_i) \log(1 - r_{\theta}(s_i | x, s_{<i}))]. \quad (4.8)$$

In practice, a separator token marks each step boundary. All non-boundary positions receive the ignore index -100 . A common three-class formulation uses class indices 0, 1, 2 for incorrect, neutral, and correct.

```

1  import torch.nn as nn
2  import torch.nn.functional as F
3
4  class ProcessRM(nn.Module):
5      def __init__(self, backbone, num_classes=3):
6          super().__init__()
7          self.backbone = backbone
8          self.step_head = nn.Linear(
9              backbone.config.hidden_size, num_classes
10         )
11
12     def forward(self, input_ids, attention_mask, labels=None):
13         out = self.backbone(
14             input_ids=input_ids,
15             attention_mask=attention_mask,
16             output_hidden_states=True,
17             return_dict=True,
18         )
19         logits = self.step_head(out.hidden_states[-1]) # [B, T, 3]

```

```

20
21     if labels is None:
22         return {"logits": logits}
23
24     # labels: 0=incorrect, 1=neutral, 2=correct at boundaries;
25     #         -100 at every other token.
26     loss = F.cross_entropy(
27         logits.reshape(-1, logits.size(-1)),
28         labels.reshape(-1),
29         ignore_index=-100,
30     )
31     return {"loss": loss, "logits": logits}

```

Listing 4.3: Three-class process reward model.

For search or reranking, PRM outputs are read only at step boundaries. Step scores can be averaged, summed with position-dependent weights, or reduced by the minimum to reject a trace as soon as one step is judged invalid.

4.1.4 Reward Models versus Value Functions

The output shape alone does not define the model. The supervision target and its semantics are decisive:

Model	Prediction	Supervision	Output location
BT RM	Relative quality $r_\theta(x, y)$	Pairwise preferences	EOS/last token
ORM	Probability of eventual correctness	Fixed offline outcome label	Every completion token
PRM	Validity of the current reasoning step	Step-level annotations	Step boundaries
Value function	Expected remaining return $V^\pi(s_t)$	Return targets from policy rollouts	Every policy token

A value function predicts

$$V^\pi(s_t) = \mathbb{E}_\pi \left[\sum_{k=t}^T \gamma^{k-t} r_k \mid s_t \right]. \quad (4.9)$$

Its targets generally depend on the current policy’s rollouts and change as that policy changes. By contrast, an ORM is normally trained offline against fixed correctness labels broadcast to token prefixes. Even if both use the same backbone and scalar per-token head, the ORM estimates outcome correctness, whereas the value head estimates remaining return and serves as a baseline for advantages such as $A_t = \hat{R}_t - V_\phi(s_t)$.

Training a sequence-level BT model on pairs formed by one correct and one incorrect response does *not* turn it into an ORM. If the objective is (4.4) and the output is one scalar per sequence, it remains a pairwise preference RM; correctness merely determines how the pairs were constructed.

4.1.5 Inference Interfaces

BT RM. Input (x, y) ; output one scalar $r_\theta(x, y)$. Use it as a terminal RL reward or rank N sampled completions by their scalar scores.

ORM. Input (x, y) ; output $p_t \approx P(\text{final answer correct} \mid x, y_{\leq t})$ at completion positions. Aggregate token scores to verify or rerank finished candidates.

PRM. Input $(x, s_1, \dots, s_K)$; output correctness scores at step boundaries. Aggregate across steps for reranking, or use scores online to prune low-quality search branches.

Value function. Input the current prefix state $s_t = (x, y_{<t})$; output $V_\phi(s_t)$. During RL training it provides a per-token baseline rather than an interpretable probability that the whole response is correct.

Chapter 5

Distillation

Off-policy distillation trains the student on sequences generated by another source, such as human-annotated data or outputs produced in advance by a teacher model. Classical knowledge distillation is typically off-policy: the student learns to match the teacher’s output distribution on teacher-generated trajectories. Informally, the student learns from model answers written by others.

On-policy distillation instead uses trajectories sampled online from the student’s current policy. The teacher provides token-level supervision on these student-generated trajectories. After each student update, new trajectories are sampled from the updated policy. In this setting, the student produces its own answers, and the teacher evaluates how it should proceed.

This distinction is important because it affects **exposure bias**. Under off-policy distillation, the student is trained primarily on high-quality teacher or reference trajectories. During inference, however, it conditions on its own outputs. An early mistake may move the student into a state rarely represented in the training data, causing subsequent errors to accumulate. Exposure bias thus arises partly from the mismatch between the teacher/reference distribution observed during training and the student-induced state distribution encountered during generation.

On-policy training reduces this mismatch by training on trajectories produced by the current student policy. Even erroneous states encountered during rollout become training examples, allowing the teacher to provide supervision at states the student is likely to visit. This better aligns the training and inference distributions, thereby mitigating exposure bias and compounding errors.

On-policy training does not, however, eliminate exposure bias entirely. Differences in teacher and student capabilities, low-quality rollouts, distributional drift over long sequences, and optimization instability may still create discrepancies between training and inference. More precisely, on-policy distillation mitigates exposure bias at the data-distribution level rather than removing it at its source. Its central advantage over purely off-policy distillation is that the student learns not only what the teacher would do along ideal trajectories, but also what the teacher recommends in states reached by the student itself.

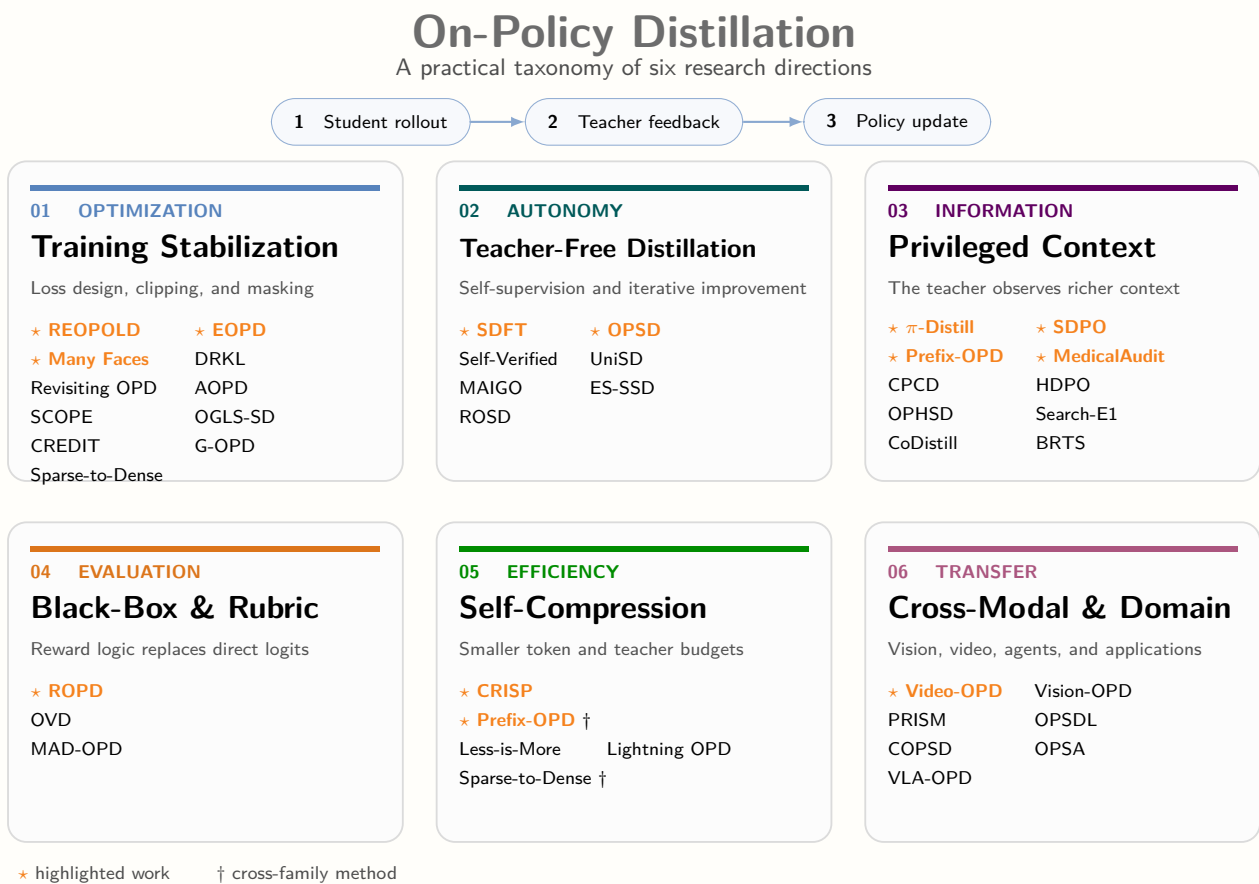


Figure 5.1: Six research directions in on-policy distillation, organized by the main source of innovation.

5.1 On-Policy Distillation: Basics

On-policy distillation (OPD) computes supervision on trajectories sampled from the current student policy π_θ . Given a prompt $x \sim \mathcal{D}_x$, the student samples a response

$$\begin{aligned} y &\sim \pi_\theta(\cdot | x), & T(y) &:= |y| \text{ (the length of the response),} \\ y &= (y_1, \dots, y_{T(y)}), \end{aligned}$$

where the end-of-sequence token is included in the response. Both the student and the fixed teacher π_{Teacher} are evaluated on each student-generated prefix $y_{<t}$. Their next-token distributions are, for all $v \in \mathcal{V}$ (the vocabulary),

$$p_t(v) := \pi_\theta(v | x, y_{<t}), \quad \text{and} \quad q_t(v) := \pi_{\text{Teacher}}(v | x, y_{<t}).$$

A standard formulation minimizes the sequence-level reverse KL on the prompt distribution. Let $\mathcal{Y}(x)$ denote the set of complete responses to x , and define the joint distribution μ_θ by sampling $x \sim \mathcal{D}_x$ first and then $y \sim \pi_\theta(\cdot | x)$. In the discrete case, this means $\mu_\theta(x, y) = \mathcal{D}_x(x) \pi_\theta(y | x)$. Expanding the KL before combining the expectations gives

$$\begin{aligned} \mathcal{L}_{\text{OPD}}(\theta) &= \mathbb{E}_{x \sim \mathcal{D}_x} [D_{\text{KL}}(\pi_\theta(\cdot | x) \| \pi_{\text{Teacher}}(\cdot | x))] \\ &= \mathbb{E}_{x \sim \mathcal{D}_x} \left[\sum_{y \in \mathcal{Y}(x)} \pi_\theta(y | x) \log \frac{\pi_\theta(y | x)}{\pi_{\text{Teacher}}(y | x)} \right] \\ &= \mathbb{E}_{x \sim \mathcal{D}_x} \left[\mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\log \frac{\pi_\theta(y | x)}{\pi_{\text{Teacher}}(y | x)} \right] \right] \\ &= \mathbb{E}_{(x, y) \sim \mu_\theta} \left[\log \frac{\pi_\theta(y | x)}{\pi_{\text{Teacher}}(y | x)} \right]. \end{aligned} \tag{5.1}$$

The last line is the law of total expectation. It does not make x and y independent: their joint law is μ_θ , under which the response y is conditioned on the sampled prompt x .

(5.1) can be equivalently rewritten as the token-level form loss function

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{(x, y) \sim \mu_\theta} \left[\sum_{t=1}^{T(y)} D_{\text{KL}}(\pi_\theta(\cdot | x, y_{<t}) \| \pi_{\text{Teacher}}(\cdot | x, y_{<t})) \right].$$

To obtain the token-level form, fix a prompt x and abbreviate the student and teacher sequence probabilities by $P_\theta(y | x)$ and $Q_{\text{Teacher}}(y | x)$. Autoregressive factorization gives

$$P_\theta(y | x) = \prod_{t=1}^{T(y)} \pi_\theta(y_t | x, y_{<t}), \quad Q_{\text{Teacher}}(y | x) = \prod_{t=1}^{T(y)} \pi_{\text{Teacher}}(y_t | x, y_{<t}). \tag{5.2}$$

Substituting these products into the definition of KL and using $\log \prod_t a_t = \sum_t \log a_t$ yields

$$\begin{aligned} D_{\text{KL}}(P_\theta(\cdot | x) \| Q_{\text{Teacher}}(\cdot | x)) &= \sum_y P_\theta(y | x) \log \frac{P_\theta(y | x)}{Q_{\text{Teacher}}(y | x)} = \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\log \frac{\prod_{t=1}^{T(y)} \pi_\theta(y_t | x, y_{<t})}{\prod_{t=1}^{T(y)} \pi_{\text{Teacher}}(y_t | x, y_{<t})} \right] \\ &= \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\sum_{t=1}^{T(y)} \log \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\text{Teacher}}(y_t | x, y_{<t})} \right]. \end{aligned} \tag{5.3}$$

For a fixed x and a realized prefix $y_{<t}$, define the corresponding conditional reverse KL by

$$\begin{aligned} \text{kl}_t(x, y_{<t}) &:= \mathbb{E}_{v \sim \pi_\theta(\cdot | x, y_{<t})} \left[\log \frac{\pi_\theta(v | x, y_{<t})}{\pi_{\text{Teacher}}(v | x, y_{<t})} \right] \\ &= D_{\text{KL}}(\pi_\theta(\cdot | x, y_{<t}) \| \pi_{\text{Teacher}}(\cdot | x, y_{<t})). \end{aligned} \tag{5.4}$$

Assume that generation is capped at $T_{\max}$. For each t , the indicator $\mathbf{1}\{t \leq T(y)\}$ is determined by $y_{<t}$ because it only records whether an end-of-sequence token has already appeared. Therefore it can be pulled through the conditional expectation over the next token. Using the tower property term by term gives

$$\begin{aligned}
& \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\sum_{t=1}^{T(y)} \log \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\text{Teacher}}(y_t | x, y_{<t})} \right] \\
&= \sum_{t=1}^{T_{\max}} \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\mathbf{1}\{t \leq T(y)\} \log \frac{\pi_\theta(y_t | x, y_{<t})}{\pi_{\text{Teacher}}(y_t | x, y_{<t})} \right] \\
&= \sum_{t=1}^{T_{\max}} \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\mathbf{1}\{t \leq T(y)\} \mathbb{E}_{v \sim \pi_\theta(\cdot | x, y_{<t})} \left[\log \frac{\pi_\theta(v | x, y_{<t})}{\pi_{\text{Teacher}}(v | x, y_{<t})} \right] \right] \\
&= \mathbb{E}_{y \sim \pi_\theta(\cdot | x)} \left[\sum_{t=1}^{T(y)} \text{kl}_t(x, y_{<t}) \right]. \tag{5.5}
\end{aligned}$$

The second equality follows by conditioning each outer expectation on the realized prefix $y_{<t}$. Conditional on x and $y_{<t}$, the actual next token has distribution $y_t \sim \pi_\theta(\cdot | x, y_{<t})$. Its conditional mean is therefore $\mathbb{E}_{v \sim \pi_\theta(\cdot | x, y_{<t})} \left[\log \frac{\pi_\theta(v | x, y_{<t})}{\pi_{\text{Teacher}}(v | x, y_{<t})} \right]$, where v is only a dummy variable for the next token and the resulting quantity depends only on $y_{<t}$; averaging it under the complete-response distribution $y \sim \pi_\theta(\cdot | x)$ simply averages over the student-induced prefixes. The unobserved suffix integrates to one. Combining (5.3), (5.4), and (5.5), then restoring the prompt expectation, yields

$$\mathcal{L}_{\text{OPD}}(\theta) = \mathbb{E}_{(x, y) \sim \mu_\theta} \left[\sum_{t=1}^{T(y)} D_{\text{KL}}(\pi_\theta(\cdot | x, y_{<t}) \| \pi_{\text{Teacher}}(\cdot | x, y_{<t})) \right]. \tag{5.6}$$

5.1.1 Implemented OPD Objectives

In practice, the actual loss computation varies with different implementations on how this exact per-token reverse KL is computed. We will see some examples below.

- **Sampled-token OPD.** The lightest variant evaluates the log-ratio only at the token y_t sampled by the student. At a realized prefix $(x, y_{<t})$, define

$$\ell_t^{\text{sample}}(x, y_{\leq t}) := \log p_t(y_t) - \log q_t(y_t). \tag{5.7}$$

Its token-level objective is

$$\mathcal{L}_{\text{OPD}}^{\text{sample}}(\theta) = \mathbb{E}_{(x, y) \sim \mu_\theta} \left[\sum_{t=1}^{T(y)} \ell_t^{\text{sample}}(x, y_{\leq t}) \right]. \tag{5.8}$$

Since $y_t \sim p_t$ conditional on $(x, y_{<t})$,

$$\mathbb{E}_{y_t \sim p_t} \left[\ell_t^{\text{sample}}(x, y_{\leq t}) \right] = \sum_{v \in \mathcal{V}} p_t(v) \log \frac{p_t(v)}{q_t(v)} = D_{\text{KL}}(p_t \| q_t). \tag{5.9}$$

Thus, the sampled-token loss is an unbiased one-sample estimator of the scalar token-level reverse KL at the sampled prefix.

- **Full-vocabulary OPD.** This variant evaluates the divergence over the entire vocabulary at every sampled prefix:

$$\mathcal{L}_{\text{OPD}}^{\text{full}}(\theta) = \mathbb{E}_{(x, y) \sim \mu_\theta} \left[\sum_{t=1}^{T(y)} D_{\text{KL}}(p_t \| q_t) \right]. \tag{5.10}$$

It provides dense supervision but materializing both distributions costs $O(BT_{\max}\mathcal{V})$ (where $|\mathcal{V}|$ is the vocabulary size) memory for batch size B .

- **Student-top- k OPD.** This intermediate variant restricts the divergence to the k tokens with highest student probability:

$$S_t := S_t(x, y_{<t}) := \text{TopK}(p_t, k) \subseteq \mathcal{V}.$$

Both distributions are renormalized on this same support:

$$\bar{p}_t^{(S_t)}(v) := \frac{p_t(v)\mathbf{1}\{v \in S_t\}}{\sum_{u \in S_t} p_t(u)}, \quad \bar{q}_t^{(S_t)}(v) := \frac{q_t(v)\mathbf{1}\{v \in S_t\}}{\sum_{u \in S_t} q_t(u)}. \quad (5.11)$$

The resulting objective is

$$\mathcal{L}_{\text{OPD}}^{\text{top-}k}(\theta) = \mathbb{E}_{(x,y) \sim \mu_\theta} \left[\sum_{t=1}^{T(y)} D_{\text{KL}}\left(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)}\right) \right]. \quad (5.12)$$

Remark.

- Probability mass outside S_t is discarded, so this is generally not equal to the full-vocabulary reverse KL. It preserves multi-token supervision on the student's high-probability region at lower cost. In other words, the sampled-token OPD is unbiased yet suffers from high variance while the full-vocabulary OPD is unbiased and has lower variance, yet it requires more memory. Top- k OPD is a compromise between the two, it is biased but has lower variance than sampled-token OPD. It also loses the probability mass information outside S_t .
- **(5.12) is ill-defined when $k = 1$.** When $k = 1$, this objective seems to degenerate. If $S_t = \{v_t^*\}$, then both restricted distributions become point masses after renormalization $\bar{p}_t(v_t^*) = 1, \bar{q}_t(v_t^*) = 1$, so the KL is identically zero. This would provide no distillation signal.
- **When is top- k OPD a good surrogate for full-vocabulary OPD?** Define the mass each distribution places on the student top- k set, $P_{S_t} = \sum_{v \in S_t} p_t(v)$ and $Q_{S_t} = \sum_{v \in S_t} q_t(v)$. Write $S_t^c = \mathcal{V} \setminus S_t$ for the complement, and assume $0 < P_{S_t}, Q_{S_t} < 1$ so that both pieces below are proper distributions. Renormalize on the complement in the same way as (5.11):

$$p_t^{S_t^c}(v) := \frac{p_t(v)\mathbf{1}\{v \in S_t^c\}}{1 - P_{S_t}}, \quad q_t^{S_t^c}(v) := \frac{q_t(v)\mathbf{1}\{v \in S_t^c\}}{1 - Q_{S_t}}. \quad (5.13)$$

Partition the reverse KL into the top- k support and its complement:

$$\begin{aligned} D_{\text{KL}}(p_t \parallel q_t) &= \sum_{v \in \mathcal{V}} p_t(v) \log \frac{p_t(v)}{q_t(v)} \\ &= \sum_{v \in S_t} p_t(v) \log \frac{p_t(v)}{q_t(v)} + \sum_{v \in S_t^c} p_t(v) \log \frac{p_t(v)}{q_t(v)}. \end{aligned} \quad (5.14)$$

On S_t , the renormalization (5.11) says $p_t(v) = P_{S_t} \bar{p}_t^{(S_t)}(v)$ and $q_t(v) = Q_{S_t} \bar{q}_t^{(S_t)}(v)$. Substituting and using $\sum_{v \in S_t} \bar{p}_t^{(S_t)}(v) = 1$ separates the shape of the conditional distribution from the mass on the set:

$$\begin{aligned} \sum_{v \in S_t} p_t(v) \log \frac{p_t(v)}{q_t(v)} &= \sum_{v \in S_t} P_{S_t} \bar{p}_t^{(S_t)}(v) \log \frac{P_{S_t} \bar{p}_t^{(S_t)}(v)}{Q_{S_t} \bar{q}_t^{(S_t)}(v)} \\ &= P_{S_t} \sum_{v \in S_t} \bar{p}_t^{(S_t)}(v) \left(\log \frac{\bar{p}_t^{(S_t)}(v)}{\bar{q}_t^{(S_t)}(v)} + \log \frac{P_{S_t}}{Q_{S_t}} \right) \\ &= P_{S_t} D_{\text{KL}}\left(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)}\right) + P_{S_t} \log \frac{P_{S_t}}{Q_{S_t}}. \end{aligned} \quad (5.15)$$

The same substitution on the complement, $p_t(v) = (1 - P_{S_t})p_t^{S_t^c}(v)$ and $q_t(v) = (1 - Q_{S_t})q_t^{S_t^c}(v)$, gives

$$\sum_{v \in S_t^c} p_t(v) \log \frac{p_t(v)}{q_t(v)} = (1 - P_{S_t}) D_{\text{KL}}(p_t^{S_t^c} \| q_t^{S_t^c}) + (1 - P_{S_t}) \log \frac{1 - P_{S_t}}{1 - Q_{S_t}}. \quad (5.16)$$

The two scalar terms are the KL between Bernoulli random variables that only record whether the next token falls in S_t . If $Z \sim \text{Bern}(P_{S_t})$ and $W \sim \text{Bern}(Q_{S_t})$, then

$$D_{\text{KL}}(Z \| W) = P_{S_t} \log \frac{P_{S_t}}{Q_{S_t}} + (1 - P_{S_t}) \log \frac{1 - P_{S_t}}{1 - Q_{S_t}}. \quad (5.17)$$

Adding (5.15) and (5.16), and identifying (5.17), produces the decomposition

$$\begin{aligned} D_{\text{KL}}(p_t \| q_t) &= P_{S_t} D_{\text{KL}}(\bar{p}_t^{(S_t)} \| \bar{q}_t^{(S_t)}) + (1 - P_{S_t}) D_{\text{KL}}(p_t^{S_t^c} \| q_t^{S_t^c}) \\ &\quad + D_{\text{KL}}(\text{Bern}(P_{S_t}) \| \text{Bern}(Q_{S_t})). \end{aligned} \quad (5.18)$$

Top- k OPD keeps only the renormalized divergence on S_t , and it drops the weight P_{S_t} . The other two terms are the bias. The complement term vanishes as $P_{S_t} \rightarrow 1$, which is the regime in which the student's top- k set already carries essentially all of its mass. The Bernoulli term is small when $P_{S_t} \approx Q_{S_t}$, i.e., when the teacher places a similar fraction of its mass on that same set. In that regime $P_{S_t} \approx 1$ as well, so dropping the weight changes the objective by little. Outside it, top- k OPD ignores both how the leftover mass is shaped and how much total mass each model assigns to S_t .

5.1.2 Gradients of the OPD Objective

The three variants above describe what is *computed*; this section describes what is *differentiated*. We first derive the exact gradient of (5.6), then show that standard OPD keeps only part of it, and finally check, variant by variant, whether plain backpropagation produces that part.

Throughout, the teacher is fixed, so $\nabla_{\theta} q_t = 0$. We use one identity repeatedly: for any probability $\pi_{\theta}(z) > 0$ that is differentiable in θ , the chain rule gives

$$\pi_{\theta}(z) \nabla_{\theta} \log \pi_{\theta}(z) = \pi_{\theta}(z) \frac{\nabla_{\theta} \pi_{\theta}(z)}{\pi_{\theta}(z)} = \nabla_{\theta} \pi_{\theta}(z). \quad (5.19)$$

We also use the stop-gradient operator $\text{sg}[\cdot]$ (`.detach()` in code), with the following convention. sg appears only inside an expression that is still to be differentiated, i.e., a loss, a surrogate, or an advantage that enters a loss, where it marks a factor that is treated as a constant under ∇_{θ} . An expression that already contains ∇_{θ} never carries sg : every factor outside ∇_{θ} is simply a value at the current θ .

The exact gradient. In (5.6), θ enters in two places: inside each per-state KL loss $\text{kl}_t(x, y_{<t})$ defined in (5.4), through $\pi_{\theta}(\cdot | x, y_{<t})$, and through μ_{θ} , which determines which prefixes $y_{<t}$ are visited. The prompt distribution $\mathcal{D}_x$ does not depend on θ , so fix x and differentiate

$$\begin{aligned} J_x(\theta) &:= \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} \left[\sum_{t=1}^{T(y)} D_{\text{KL}}(\pi_{\theta}(\cdot | x, y_{<t}) \| \pi_{\text{Teacher}}(\cdot | x, y_{<t})) \right] \\ &= \mathbb{E}_{y \sim \pi_{\theta}(\cdot | x)} \left[\sum_{t=1}^{T(y)} \text{kl}_t(x, y_{<t}) \right]. \end{aligned}$$

1. *Reduce each term to a sum over prefixes.* As in the derivation of (5.5), cap generation at $T_{\max}$ and use the indicator $\mathbf{1}\{t \leq T(y)\}$, which is determined by $y_{<t}$. The t -th summand then depends on y only through $y_{<t}$, so its expectation only needs the marginal probability of the prefix, $\pi_{\theta}(y_{<t} | x) = \prod_{s=1}^{t-1} p_s(y_s)$:

$$J_x(\theta) = \sum_{t=1}^{T_{\max}} \sum_{y_{<t}} \pi_{\theta}(y_{<t} | x) \mathbf{1}\{t \leq T(y)\} \text{kl}_t(x, y_{<t}).$$

2. *Product rule.* Every sum is finite, so the gradient passes inside. Each summand is a product of two θ -dependent factors, $\pi_{\theta}(y_{<t} | x)$ and $\text{kl}_t(x, y_{<t})$; the indicator does not depend on θ . Applying the product

rule, then (5.19) to $\nabla_{\theta} \pi_{\theta}(y_{<t} \mid x)$,

$$\begin{aligned} \nabla_{\theta} J_x(\theta) &= \sum_{t=1}^{T_{\max}} \sum_{y_{<t}} \mathbf{1}\{t \leq T(y)\} [\pi_{\theta}(y_{<t} \mid x) \nabla_{\theta} \text{kl}_t(x, y_{<t}) + \text{kl}_t(x, y_{<t}) \nabla_{\theta} \pi_{\theta}(y_{<t} \mid x)] \\ &= \sum_{t=1}^{T_{\max}} \sum_{y_{<t}} \pi_{\theta}(y_{<t} \mid x) \mathbf{1}\{t \leq T(y)\} [\nabla_{\theta} \text{kl}_t(x, y_{<t}) + \text{kl}_t(x, y_{<t}) \nabla_{\theta} \log \pi_{\theta}(y_{<t} \mid x)]. \end{aligned}$$

Undoing step 1 (the bracket still depends on y only through $y_{<t}$) turns this back into an expectation over complete responses:

$$\nabla_{\theta} J_x(\theta) = \mathbb{E}_{y \sim \pi_{\theta}(\cdot \mid x)} \left[\sum_{t=1}^{T(y)} \nabla_{\theta} \text{kl}_t(x, y_{<t}) \right] + \mathbb{E}_{y \sim \pi_{\theta}(\cdot \mid x)} \left[\sum_{t=1}^{T(y)} \text{kl}_t(x, y_{<t}) \nabla_{\theta} \log \pi_{\theta}(y_{<t} \mid x) \right].$$

3. *Expand the prefix score.* Taking the log of $\pi_{\theta}(y_{<t} \mid x) = \prod_{s=1}^{t-1} p_s(y_s)$ gives $\nabla_{\theta} \log \pi_{\theta}(y_{<t} \mid x) = \sum_{s=1}^{t-1} \nabla_{\theta} \log p_s(y_s)$. The second expectation therefore contains the double sum

$$\sum_{t=1}^{T(y)} \sum_{s=1}^{t-1} \text{kl}_t(x, y_{<t}) \nabla_{\theta} \log p_s(y_s).$$

4. *Swap the order of summation.* The double sum runs over all pairs $1 \leq s < t \leq T(y)$. Summing over s on the outside instead,

$$\sum_{t=1}^{T(y)} \sum_{s=1}^{t-1} \text{kl}_t(x, y_{<t}) \nabla_{\theta} \log p_s(y_s) = \sum_{s=1}^{T(y)} \nabla_{\theta} \log p_s(y_s) \sum_{t=s+1}^{T(y)} \text{kl}_t(x, y_{<t}).$$

Renaming $s \rightarrow t$ and $t \rightarrow t'$ and restoring the expectation over $x \sim \mathcal{D}_x$ gives

$$\nabla_{\theta} \mathcal{L}_{\text{OPD}}(\theta) = \underbrace{\mathbb{E}_{(x,y) \sim \mu_{\theta}} \left[\sum_{t=1}^{T(y)} \nabla_{\theta} \text{kl}_t(x, y_{<t}) \right]}_{\text{(A): current state}} + \underbrace{\mathbb{E}_{(x,y) \sim \mu_{\theta}} \left[\sum_{t=1}^{T(y)} \nabla_{\theta} \log p_t(y_t) \sum_{t'=t+1}^{T(y)} \text{kl}_{t'}(x, y_{<t'}) \right]}_{\text{(B): future states}}. \quad (5.20)$$

Term (A) changes $\pi_{\theta}(\cdot \mid x, y_{<t})$ to reduce the KL at the state already visited. Term (B) is a REINFORCE term whose “reward” for token y_t is the total KL at all later positions: y_t determines which future states are visited, so it is credited with their KL. Note that kl_t itself does not appear in the reward of y_t , because $\text{kl}_t(x, y_{<t})$ does not depend on y_t .

What standard OPD keeps. Standard OPD drops term (B): each visited state $(x, y_{<t})$ is treated as a constant and only term (A) is kept. In reinforcement-learning terms, this corresponds to a discount factor of zero. The three variants differ only in how they compute (or estimate) $\nabla_{\theta} \text{kl}_t(x, y_{<t})$, or its top- k analogue, at a fixed state. Below, write $\delta_t(v) := \log p_t(v) - \log q_t(v)$, so that $\text{kl}_t(x, y_{<t}) = \sum_{v \in \mathcal{V}} p_t(v) \delta_t(v)$.

Full-vocabulary and top- k OPD: plain backpropagation is correct. The full-vocabulary variant computes $\text{kl}_t(x, y_{<t}) = \sum_{v \in \mathcal{V}} p_t(v) \delta_t(v)$ exactly. The top- k variant computes $D_{\text{KL}}(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)})$; the renormalized log-probabilities are obtained by subtracting a log-sum-exp, $\log p_t(i) - \log \sum_{j \in S_t} p_t(j) = \log \bar{p}_t^{(S_t)}(i)$ by (5.11), and likewise for the teacher. The index set S_t is chosen from detached log-probabilities, so it is held fixed; this is correct because S_t is piecewise constant in θ . In both variants the value is a deterministic, differentiable function of θ at a fixed state, with no sampling inside, so autograd returns its exact gradient. For the top- k variant this is the exact gradient of the top- k objective, which is itself a biased stand-in for kl_t (see the remark after (5.12)).

Sampled-token OPD: plain backpropagation is wrong. Here the computed value $\ell_t^{\text{sample}} = \delta_t(y_t)$ depends on the token $y_t \sim p_t$, which is drawn from the distribution being trained. Its expectation is the target, $\mathbb{E}_{y_t \sim p_t}[\delta_t(y_t)] = \text{kl}_t(x, y_{<t})$ by (5.9), but autograd cannot differentiate through the sampling step. The argument is a one-step version of the derivation above.

1. *What autograd computes.* Since $\nabla_{\theta} q_t = 0$, backpropagating ℓ_t^{sample} gives $\nabla_{\theta} \delta_t(y_t) = \nabla_{\theta} \log p_t(y_t)$. The teacher does not appear.

2. *Its expectation is zero.* Because $\mathcal{V}$ is finite, the expectation is a finite sum, so

$$\mathbb{E}_{y_t \sim p_t} [\nabla_{\theta} \log p_t(y_t)] = \sum_{v \in \mathcal{V}} p_t(v) \nabla_{\theta} \log p_t(v) \stackrel{(5.19)}{=} \sum_{v \in \mathcal{V}} \nabla_{\theta} p_t(v) = \nabla_{\theta} \sum_{v \in \mathcal{V}} p_t(v) = \nabla_{\theta} 1 = 0 \quad (5.21)$$

So the update is zero-mean noise that carries no information from the teacher.

3. *The true gradient.* Applying the product rule inside the finite sum,

$$\begin{aligned} \nabla_{\theta} \text{kl}_t(x, y_{<t}) &= \nabla_{\theta} \sum_{v \in \mathcal{V}} p_t(v) \delta_t(v) = \sum_{v \in \mathcal{V}} \nabla_{\theta} p_t(v) \delta_t(v) + \sum_{v \in \mathcal{V}} p_t(v) \nabla_{\theta} \delta_t(v) \\ &\stackrel{(5.19)}{=} \underbrace{\sum_{v \in \mathcal{V}} p_t(v) \delta_t(v) \nabla_{\theta} \log p_t(v)}_{\mathbb{E}_{y_t \sim p_t} [\delta_t(y_t) \nabla_{\theta} \log p_t(y_t)]} + \underbrace{\sum_{v \in \mathcal{V}} p_t(v) \nabla_{\theta} \log p_t(v)}_{= 0 \text{ by (5.21)}}. \end{aligned}$$

All of the signal is in the first sum, which comes from differentiating the sampling distribution. We record this as

$$\nabla_{\theta} \text{kl}_t(x, y_{<t}) = \sum_{v \in \mathcal{V}} p_t(v) \delta_t(v) \nabla_{\theta} \log p_t(v). \quad (5.22)$$

4. *The surrogate.* The pseudocode below therefore returns the loss as $\text{sg}[\delta_t(y_t)] \log p_t(y_t)$ instead of $\delta_t(y_t)$. Its value has no meaning, but its gradient is $\delta_t(y_t) \nabla_{\theta} \log p_t(y_t)$, whose expectation over $y_t \sim p_t$ is the first sum in step 3, i.e., $\nabla_{\theta} \text{kl}_t(x, y_{<t})$. This is the score-function (REINFORCE) estimator, and it is the same construction as A_t^{OPD} in PG-OPD implemented in VeRL (discussed below).

5.1.3 Pseudocode

The pseudocode below implements the three variants. In `full` and `topk`, the returned value is the per-state divergence itself; in `sampld`, it is the surrogate from Section 5.1.2.

```

1  def opd_token_loss(
2      student_logits,
3      teacher_logits,
4      mode="full",
5      sampled_tokens=None,
6      top_k=None,
7  ):
8      """Return one OPD loss per batch element and response position."""
9      s_logp = student_logits.float().log_softmax(dim=-1)
10     with torch.no_grad():
11         t_logp = teacher_logits.float().log_softmax(dim=-1)
12
13     if mode == "sampled":
14         if sampled_tokens is None:
15             raise ValueError("sampled_tokens is required")
16         token_idx = sampled_tokens.unsqueeze(dim=-1)
17         s_token = s_logp.gather(dim=-1, index=token_idx).squeeze(dim=-1)
18         t_token = t_logp.gather(dim=-1, index=token_idx).squeeze(dim=-1)
19         # Not the loss value itself; see the gradient discussion above.
20         return (s_token - t_token).detach() * s_token
21
22     if mode == "full":
23         return (s_logp.exp() * (s_logp - t_logp)).sum(dim=-1)
24
25     if mode == "topk":
26         if top_k is None:
27             raise ValueError("top_k is required")
28
29         # Student-selected support S_t.
30         idx = s_logp.detach().topk(top_k, dim=-1).indices
31         s_top = s_logp.gather(dim=-1, index=idx)
32         t_top = t_logp.gather(dim=-1, index=idx)
33
34         # Renormalize student and teacher on the same support.
35         s_bar = s_top - s_top.logsumexp(dim=-1, keepdim=True)

```

```

36         t_bar = t_top - t_top.logsumexp(dim=-1, keepdim=True)
37         return (s_bar.exp() * (s_bar - t_bar)).sum(dim=-1)
38
39     raise ValueError(f"unknown OPD mode: {mode}")
40
41
42 teacher = load_teacher() # A model stronger than the student
43 student = init_student()
44 optimizer = AdamW(student.parameters(), lr=1e-6)
45
46 for step in range(total_steps):
47     prompt = sample(prompts)
48
49     # The student generates the states on which distillation is performed.
50     with torch.no_grad():
51         response_tokens = student.generate(prompt, max_tokens=max_len)
52
53     trajectory = concatenate(prompt, response_tokens)
54     s_all_logits = student(trajectory).logits
55     with torch.no_grad():
56         t_all_logits = teacher(trajectory).logits
57
58     # Select next-token logits aligned with y_1, ..., y_{T(y)} only.
59     s_logits = response_position_logits(s_all_logits, prompt)
60     t_logits = response_position_logits(t_all_logits, prompt)
61
62     # Three alternatives; choose one for a training run.
63     sampled_loss = opd_token_loss(s_logits, t_logits, mode="sampled", sampled_tokens=
        response_tokens)
64     # full_loss = opd_token_loss(s_logits, t_logits, mode="full")
65     # topk_loss = opd_token_loss(s_logits, t_logits, mode="topk", top_k=64)
66
67     # Exclude padding after each response's EOS token.
68     active = response_activity_mask(response_tokens)
69     loss = (full_loss * active).sum() / active.sum()
70     optimizer.zero_grad()
71     loss.backward()
72     optimizer.step()

```

5.2 OPD Implementation Discussions

5.2.1 Original VeRL

VeRL has two paths for you to use OPD:

- **PG-OPD:** The negative distillation loss is used as the token-level advantage estimation in the PPO loss. Specifically, for the sampled student token y_t , construct the detached token-level signal:

$$A_t^{\text{OPD}} = -\text{sg}[\log p_t(y_t) - \log q_t(y_t)],$$

where sg is the stop-gradient operator, $p_t(v) := \pi_\theta(v \mid x, y_{<t})$, and $q_t(v) := \pi_{\text{Teacher}}(v \mid x, y_{<t})$. Then feed it into the existing [PPO policy-loss calculation](#).

- **GKD-OPD:** The distillation loss is the target. However, the VeRL implements the R. Agarwal et al. (2024) so the forward-KL is used, i.e., when using the top-k to do the KL estimation, the teacher-top-k support is used.

5.2.1.1 OPD as a PPO Update

The VeRL actor is built for PPO. Given responses sampled from the rollout policy $\pi_{\theta_{\text{old}}}$ and one advantage $\hat{A}_t$ per response position, it minimizes the clipped objective, aggregated over response positions according to `loss_agg_mode`,

$$\max \left\{ -\hat{A}_t w_t(\theta), -\hat{A}_t \text{clip}(w_t(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \right\}, \quad w_t(\theta) := \frac{p_t(y_t)}{p_t^{\text{old}}(y_t)}, \quad (5.23)$$

where $p_t^{\text{old}}(v) := \pi_{\theta_{\text{old}}}(v \mid x, y_{<t})$. PG-OPD reuses this objective unchanged and only supplies the advantage: in the [code](#), `advantages=distillation_losses.detach()`, where `distillation_losses` is the per-token log-ratio

(the k1 estimator). In the notation of Section 5.1.2,

$$\hat{A}_t = A_t^{\text{OPD}} = -\text{sg}[\delta_t(y_t)].$$

The update loop. Each PPO iteration of the `trainer` samples responses with the current parameters θ_{old} , computes `old_log_probs` once, i.e., $\log p_t^{\text{old}}(y_t)$, and then calls the actor update. The update (`train_mini_batch`) splits the batch into mini-batches, passes over them `epochs` times, and takes one optimizer step per mini-batch. Denote the parameters at these successive steps by $\theta^{(0)} = \theta_{\text{old}}, \theta^{(1)}, \theta^{(2)}, \dots$. At the next PPO iteration, θ_{old} is reset to the current parameters and the same pattern repeats. Consequently, $w_t(\theta) = 1$ holds only at the first optimizer step of each PPO iteration; at every later step of the same iteration, $\theta \neq \theta_{\text{old}}$.

In upstream VeRL the advantage is *not* frozen at rollout time: the `per-token distillation loss` is recomputed in every mini-batch from the current forward pass (`model_output["log_probs"]`) and then detached. So at step $\theta^{(j)}$, $\hat{A}_t = -\text{sg}[\delta_t(y_t)]$ with δ_t evaluated at $\theta^{(j)}$.

Gradient of one term. Since $p_t^{\text{old}}(y_t)$ does not depend on θ , given (5.19), one has

$$\nabla_{\theta} w_t(\theta) = \frac{\nabla_{\theta} p_t(y_t)}{p_t^{\text{old}}(y_t)} = \frac{p_t(y_t)}{p_t^{\text{old}}(y_t)} \nabla_{\theta} \log p_t(y_t) = w_t(\theta) \nabla_{\theta} \log p_t(y_t).$$

Since $\hat{A}_t$ is constant in θ with value $-\delta_t(y_t)$, the gradient of (5.23) is

$$-\hat{A}_t \nabla_{\theta} w_t(\theta) = w_t(\theta) \delta_t(y_t) \nabla_{\theta} \log p_t(y_t)$$

when the clip is not binding, and zero when the clip (or the dual-clip bound) binds, because the clipped branch is then constant in θ .

- **First optimizer step of each PPO iteration.** Here $\theta = \theta_{\text{old}}$, so $w_t = 1$ and neither clip binds, because $1 \in [1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$ and the dual-clip bound exceeds one. The gradient is $\delta_t(y_t) \nabla_{\theta} \log p_t(y_t)$, exactly the gradient of the sampled-token surrogate $\text{sg}[\delta_t(y_t)] \log p_t(y_t)$ of Section 5.1.2. Its expectation over $y_t \sim p_t$ is $\nabla_{\theta} \text{kl}_t(x, y_{<t})$, i.e., term (A) of (5.20).
- **Later optimizer steps of the same iteration.** Now $w_t \neq 1$ in general, and y_t was drawn from p_t^{old} rather than p_t . Where the clip does not bind, the ratio corrects for this exactly:

$$\mathbb{E}_{y_t \sim p_t^{\text{old}}(\cdot | x, y_{<t})} [w_t \delta_t(y_t) \nabla_{\theta} \log p_t(y_t)] = \sum_{v \in \mathcal{V}} p_t^{\text{old}}(v) \frac{p_t(v)}{p_t^{\text{old}}(v)} \delta_t(v) \nabla_{\theta} \log p_t(v) = \nabla_{\theta} \text{kl}_t(x, y_{<t}),$$

evaluated at the current θ , where the last equality is (5.22). This identity holds for a fixed prefix $(x, y_{<t})$; the expectation is over y_t only. Two things still separate the update from term (A) of (5.20) at the current θ .

1. *Prefixes.* Term (A) averages $\nabla_{\theta} \text{kl}_t(x, y_{<t})$ over prefixes drawn from π_{θ} , but these prefixes were drawn from $\pi_{\theta_{\text{old}}}$. The ratio w_t reweights only y_t , not $y_{<t}$.
2. *Clipping.* The identity uses the unclipped gradient. A token with $\hat{A}_t > 0$ and $w_t > 1 + \epsilon_{\text{high}}$, or with $\hat{A}_t < 0$ and $w_t < 1 - \epsilon_{\text{low}}$ (or beyond the dual-clip bound), contributes zero gradient, so the expected update differs from $\nabla_{\theta} \text{kl}_t(x, y_{<t})$. This bias is intentional: it is PPO's trust region, which stops pushing a token's probability once it has moved far enough from the rollout policy.

Dictionary between PPO and OPD.

- **Reward.** The per-token reward is $-\delta_t(y_t)$: the teacher's log-probability minus the student's at the sampled token. There is no task reward.
- **Return and discount.** The advantage is the immediate reward only, i.e., discount $\gamma = 0$. This is exactly the step of dropping term (B) in (5.20). Using the reward-to-go $\hat{A}_t = -\sum_{t' \geq t} \text{sg}[\delta_{t'}(y_{t'})]$ ($\gamma = 1$) would add an unbiased estimate of term (B), since $\mathbb{E}[\delta_{t'}(y_{t'}) | x, y_{<t'}] = \text{kl}_{t'}(x, y_{<t'})$ and $\nabla_{\theta} \log p_t(y_t)$ is fixed given $y_{<t'}$ for $t < t'$.
- **Baseline and critic.** None; the advantage is not centered or normalized.
- **Ratio and clipping.** Inactive at the first optimizer step of each PPO iteration. At the later steps of the iteration, the ratio corrects the next-token sampling and the clip keeps θ close to θ_{old} , exactly as in PPO.

Remark (Is VeRL optimizing the OPD objective?). Ignore clipping. At later optimizer steps of a PPO iteration, the expected update is not term (A) of (5.20) at the current θ . This is not a bug, for three reasons.

1. *It is the exact gradient of a frozen-state surrogate.* Define the objective with the prefix distribution frozen at the rollout policy,

$$\tilde{J}(\theta) := \mathbb{E}_{(x,y) \sim \mu_{\theta_{\text{old}}}} \left[\sum_{t=1}^{T(y)} \text{kl}_t(x, y_{<t}) \right],$$

where only kl_t , through p_t , depends on θ . Its gradient is $\mathbb{E}_{\mu_{\theta_{\text{old}}}} [\sum_t \nabla_{\theta} \text{kl}_t(x, y_{<t})]$, which by the per-prefix identity above is the expectation of the unclipped update $w_t \delta_t(y_t) \nabla_{\theta} \log p_t(y_t)$ summed over t . So, with clipping ignored, VeRL runs unbiased stochastic gradient descent on $\tilde{J}$. In expectation, $\tilde{J}$ has the same gradient as the unclipped importance-weighted surrogate $\mathbb{E}_{\mu_{\theta_{\text{old}}}} [\sum_t w_t(\theta) \hat{A}_t]$ used by TRPO; PPO optimizes a clipped version of that surrogate.

2. *The surrogate is accurate near θ_{old} .* At $\theta = \theta_{\text{old}}$, $\nabla_{\theta} \tilde{J}$ equals term (A). For general θ , the two differ only in the prefix distribution. Let $G := \sup_{(x,y)} \|\sum_{t=1}^{T(y)} \nabla_{\theta} \text{kl}_t(x, y_{<t})\|$, assumed finite (it is whenever $\mathcal{D}_x$ has finite support, since responses are capped at T_{max}). Since $\|\mathbb{E}_P[f] - \mathbb{E}_Q[f]\| \leq 2 \text{TV}(P, Q) \sup \|f\|$, Pinsker's inequality, and the chain rule for the KL divergence give

$$\begin{aligned} \|(A) - \nabla_{\theta} \tilde{J}(\theta)\| &\leq 2G \text{TV}(\mu_{\theta}, \mu_{\theta_{\text{old}}}) \leq G \sqrt{2 D_{\text{KL}}(\mu_{\theta_{\text{old}}} \parallel \mu_{\theta})}, \\ D_{\text{KL}}(\mu_{\theta_{\text{old}}} \parallel \mu_{\theta}) &= \mathbb{E}_{\mu_{\theta_{\text{old}}}} \left[\sum_{t=1}^{T(y)} D_{\text{KL}}(p_t^{\text{old}} \parallel p_t) \right]. \end{aligned}$$

The gap is therefore controlled by how far the policy has moved since the rollout, which is what a trust region (PPO's clip, or TRPO's KL constraint) is designed to keep small. The prefix mismatch is thus not a separate flaw next to the trust region; it is the reason a trust region is needed.

3. *OPD already approximates more coarsely.* Dropping term (B), i.e., using $\gamma = 0$, means that even the first optimizer step of each iteration does not follow the gradient of (5.6). Every standard OPD implementation makes this choice by design; the later optimizer steps add a second, local approximation on top of it.

If exactness with respect to term (A) is required, use a single mini-batch and a single epoch per rollout. Every update is then an unbiased estimate of term (A), at the cost of sample efficiency.

Rethinking-OPD, discussed next, uses the same machinery but replaces the single sampled token y_t by k enumerated candidates.

5.2.2 Rethink OPD modified VeRL

[Github Repo of Rethinking-OPD](#)² implements a modified version of the student top-k OPD (5.12). It keeps the student top- k set S_t , but instead of backpropagating the renormalized divergence $D_{\text{KL}}(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)})$, it runs the PPO update of Section 5.2.1.1 with one advantage per *candidate* $i \in S_t$ rather than per sampled token. We first describe and analyze the method, then map it to the code.

5.2.2.1 The Method

Throughout, $\delta_t(i) = \log p_t(i) - \log q_t(i)$ as in Section 5.1.2.

Per-candidate advantage. Fix a prefix $(x, y_{<t})$. Every variant aims at the per-state gradient $\nabla_{\theta} \text{kl}_t(x, y_{<t})$ (term (A) of (5.20)). Define the per-token advantage function $\hat{A}_t(v) := -\text{sg}[\delta_t(v)]$ for $v \in \mathcal{V}$. By (5.22), the target is an expectation under the student's next-token distribution:

$$\nabla_{\theta} \text{kl}_t(x, y_{<t}) = \sum_{v \in \mathcal{V}} p_t(v) \delta_t(v) \nabla_{\theta} \log p_t(v) = \mathbb{E}_{v \sim p_t} \left[-\hat{A}_t(v) \nabla_{\theta} \log p_t(v) \right]. \quad (5.24)$$

There are two natural ways to estimate this expectation.

- **Sample one token (PG-OPD).** Draw $y_t \sim p_t$ and use $-\hat{A}_t(y_t) \nabla_{\theta} \log p_t(y_t)$. This is a one-sample Monte Carlo estimate of (5.24): it is random, since it depends on y_t ; it is unbiased; and it has sampling variance.
- **Enumerate a candidate set (Rethinking-OPD).** Replace p_t by its renormalization $\bar{p}_t^{(S_t)}$ on S_t and compute the expectation exactly:

$$\mathbb{E}_{i \sim \bar{p}_t^{(S_t)}} \left[-\hat{A}_t(i) \nabla_{\theta} \log p_t(i) \right] = - \sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \hat{A}_t(i) \nabla_{\theta} \log p_t(i).$$

This estimate is deterministic given the prefix, so it has no variance over y_t . It equals (5.24) exactly when $S_t = \mathcal{V}$ and is biased when $|S_t| < |\mathcal{V}|$; the bias is characterized below.

The first estimate has the form “advantage times the score $\nabla_\theta \log p_t(\cdot)$ of a token”. The second estimate has the same form, once per candidate: the coefficient of $\nabla_\theta \log p_t(i)$ is

$$A_{t,i} := \bar{p}_t^{(S_t)}(i) \hat{A}_t(i) = -\text{sg}\left[\bar{p}_t^{(S_t)}(i) \delta_t(i)\right], \quad i \in S_t, \quad (5.25)$$

so that the second estimate equals $-\sum_{i \in S_t} A_{t,i} \nabla_\theta \log p_t(i)$. That is, $A_{t,i}$ is the PG-OPD advantage of token i , weighted by $\bar{p}_t^{(S_t)}(i)$. In PG-OPD this weight is supplied implicitly by sampling; here the candidates are enumerated, so it must appear explicitly. Because each candidate now looks like a sampled token with its own advantage, the PPO loss of Section 5.2.1.1 can be reused unchanged, as described next.

The loss. The PPO objective (5.23) is applied to each candidate and summed over S_t . With the per-candidate ratio $w_{t,i}(\theta) := p_t(i)/p_t^{\text{old}}(i)$, the per-state loss is

$$\sum_{i \in S_t} \max \left\{ -A_{t,i} w_{t,i}(\theta), -A_{t,i} \text{clip}(w_{t,i}(\theta), 1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}) \right\}. \quad (5.26)$$

The sampled token y_t does not appear in (5.26); it only determines which prefixes $(x, y_{<t})$ are visited. At the first optimizer step of each PPO iteration, $w_{t,i} = 1$, the clip is inactive, and $\nabla_\theta w_{t,i} = w_{t,i} \nabla_\theta \log p_t(i) = \nabla_\theta \log p_t(i)$, exactly as in Section 5.2.1.1. The gradient of (5.26) therefore equals that of the policy-gradient surrogate (“cand” for candidate)

$$\ell_t^{\text{cand}} := -\sum_{i \in S_t} A_{t,i} \log p_t(i) = \sum_{i \in S_t} \text{sg}\left[\bar{p}_t^{(S_t)}(i) \delta_t(i)\right] \log p_t(i). \quad (5.27)$$

Like the sampled-token surrogate, its value has no meaning; only its gradient matters. With S_t held fixed, that gradient is

$$g_t := \nabla_\theta \ell_t^{\text{cand}} = \sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \delta_t(i) \nabla_\theta \log p_t(i). \quad (5.28)$$

At later optimizer steps of the same iteration, the per-candidate ratio and clip act exactly as in Section 5.2.1.1. The rest of this section analyzes g_t .

Relation to the full-vocabulary gradient. Compare g_t with the exact per-state gradient (5.22), derived in step 3 of the sampled-token analysis in Section 5.1.2. Sampled-token OPD estimates (5.22) with a single draw $v = y_t \sim p_t$, so the sampling itself supplies the weight $p_t(v)$. Rethinking-OPD enumerates the candidates instead of sampling them, so the weight must appear explicitly: (5.28) is (5.22) truncated to S_t and renormalized, and it recovers (5.22) exactly when $S_t = \mathcal{V}$.

Comparison with student-top- k OPD. Write $\bar{\delta}_t(i) := \log \bar{p}_t^{(S_t)}(i) - \log \bar{q}_t^{(S_t)}(i)$ for the renormalized log-ratio, so that $D_{\text{KL}}(\bar{p}_t^{(S_t)} \| \bar{q}_t^{(S_t)}) = \sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \bar{\delta}_t(i)$. The argument of step 3 in Section 5.1.2 applies verbatim to this pair of distributions on S_t . It uses the product rule, $\nabla_\theta \bar{\delta}_t(i) = \nabla_\theta \log \bar{p}_t^{(S_t)}(i)$, (5.19), and

$$\sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \nabla_\theta \log \bar{p}_t^{(S_t)}(i) = \nabla_\theta \sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) = \nabla_\theta 1 = 0.$$

Placing the resulting top- k gradient next to the other two per-state gradients,

$$\begin{aligned} \text{full vocabulary: } \nabla_\theta \text{kl}_t(x, y_{<t}) &= \sum_{v \in \mathcal{V}} p_t(v) \delta_t(v) \nabla_\theta \log p_t(v), \\ \text{student top-}k: \nabla_\theta D_{\text{KL}}\left(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)}\right) &= \sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \bar{\delta}_t(i) \nabla_\theta \log \bar{p}_t^{(S_t)}(i), \\ \text{Rethinking-OPD: } g_t &= \sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \delta_t(i) \nabla_\theta \log p_t(i). \end{aligned} \quad (5.29)$$

Top- k OPD renormalizes everything: the weights, the log-ratio, and the score. It therefore sees only the *shape* of p_t on S_t . Rethinking-OPD renormalizes only the weights and keeps the unnormalized log-ratio and score of the full-vocabulary gradient.

To relate the last two, for $i \in S_t$, substitute $\bar{\delta}_t(i) = \delta_t(i) - c_t$ with $c_t := \log(P_{S_t}/Q_{S_t})$, where $P_{S_t} = \sum_{i \in S_t} p_t(i)$ and $Q_{S_t} = \sum_{i \in S_t} q_t(i)$ are the student and teacher masses on S_t (as in the remark after (5.12)), and

$\nabla_{\theta} \log \bar{p}_t^{(S_t)}(i) = \nabla_{\theta} \log p_t(i) - \nabla_{\theta} \log P_{S_t}$ into (5.29). Using $\sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \nabla_{\theta} \log p_t(i) = \sum_{i \in S_t} \nabla_{\theta} p_t(i) / P_{S_t} = \nabla_{\theta} \log P_{S_t}$ and $\sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) = 1$,

$$\begin{aligned} \nabla_{\theta} D_{\text{KL}}(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)}) &= \sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) (\delta_t(i) - c_t) (\nabla_{\theta} \log p_t(i) - \nabla_{\theta} \log P_{S_t}) \\ &= g_t - \mathbb{E}_{i \sim \bar{p}_t^{(S_t)}} [\delta_t(i)] \nabla_{\theta} \log P_{S_t} - c_t \nabla_{\theta} \log P_{S_t} + c_t \nabla_{\theta} \log P_{S_t}. \end{aligned}$$

The last two terms cancel, so

$$g_t = \underbrace{\nabla_{\theta} D_{\text{KL}}(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)})}_{\text{shape on } S_t} + \underbrace{\mathbb{E}_{i \sim \bar{p}_t^{(S_t)}} [\delta_t(i)] \nabla_{\theta} \log P_{S_t}}_{\text{mass on } S_t}. \quad (5.30)$$

The first term is exactly the gradient of student-top- k OPD. The second term changes only the student's total mass P_{S_t} on S_t : under gradient descent it moves mass onto S_t when $\mathbb{E}_{\bar{p}}[\delta_t] < 0$ (on average the teacher assigns more probability to the candidates than the student) and off S_t otherwise. This is the information that (5.18) shows is discarded by renormalization. At $k = 1$ the first term vanishes and only the mass term remains, which is why Rethinking-OPD does not degenerate.

What g_t descends. An update direction alone does not say what the method converges to. We therefore ask whether g_t is the gradient of some function at a fixed state. Recall the student and teacher masses on S_t from the remark after (5.12), $P_{S_t} = \sum_{i \in S_t} p_t(i)$ and $Q_{S_t} = \sum_{i \in S_t} q_t(i)$, and the on-support part of the reverse KL in (5.14),

$$F_t := \sum_{i \in S_t} p_t(i) \log \frac{p_t(i)}{q_t(i)} = \sum_{i \in S_t} p_t(i) \delta_t(i).$$

By the product rule and (5.19), with $\nabla_{\theta} q_t = 0$,

$$\nabla_{\theta} F_t = \sum_{i \in S_t} (\nabla_{\theta} p_t(i) \delta_t(i) + p_t(i) \nabla_{\theta} \log p_t(i)) = \sum_{i \in S_t} p_t(i) \delta_t(i) \nabla_{\theta} \log p_t(i) + \nabla_{\theta} P_{S_t}.$$

Over the full vocabulary the last term would be $\nabla_{\theta} 1 = 0$, but on S_t it is not. Since $\bar{p}_t^{(S_t)}(i) = p_t(i)/P_{S_t}$, the first sum equals $P_{S_t} g_t$, so

$$g_t = \frac{1}{P_{S_t}} \nabla_{\theta} \Phi_t, \quad \Phi_t := F_t - P_{S_t} = \sum_{i \in S_t} \left[p_t(i) \log \frac{p_t(i)}{q_t(i)} - p_t(i) \right]. \quad (5.31)$$

Here $1/P_{S_t}$ is evaluated at the current parameters and not differentiated: it enters through the detached weights and only rescales the step at each state. Hence, at a fixed state and with S_t fixed, Rethinking-OPD performs gradient descent on Φ_t , the on-support part of the reverse KL minus the student mass on S_t .

Minimizer of Φ_t . Each summand of (5.31) depends on a single probability $u = p_t(i)$ through $\varphi(u) = u \log(u/q_t(i)) - u$, with $\varphi'(u) = \log(u/q_t(i))$ and $\varphi''(u) = 1/u > 0$. So φ is strictly convex and minimized at $u = q_t(i)$, and Φ_t is minimized exactly when $p_t(i) = q_t(i)$ for all $i \in S_t$. This is attainable, since $Q_{S_t} \leq 1$ leaves mass $1 - Q_{S_t}$ for the complement. Unlike (5.12), the method therefore matches the teacher's *absolute* probabilities on S_t , not only their shape.¹ At $k = 1$, $\Phi_t = \varphi(p_t(v_t^*))$ with $S_t = \{v_t^*\}$, minimized at $p_t(v_t^*) = q_t(v_t^*)$, whereas (5.12) is identically zero. At the level of objectives, (5.15) gives $\Phi_t = P_{S_t} D_{\text{KL}}(\bar{p}_t^{(S_t)} \parallel \bar{q}_t^{(S_t)}) + [P_{S_t} \log(P_{S_t}/Q_{S_t}) - P_{S_t}]$; the bracket, as a function of P_{S_t} , is minimized at $P_{S_t} = Q_{S_t}$, which is the objective-level counterpart of the mass term in (5.30).

Are the weights necessary? In sampled-token OPD the advantage $-\text{sg}[\delta_t(y_t)]$ carries no probability weight, so the weight $\bar{p}_t^{(S_t)}(i)$ in (5.25) may look redundant. This is exactly the question raised in [Rethinking-OPD issue #13²](#), which proposes $A_{t,i} = -\text{sg}[\delta_t(i)]$ (`rm_scores = -kl_val`). The weight is necessary: the candidates are enumerated rather than sampled, so nothing else supplies the factor $p_t(v)$ of (5.22). Without it the gradient becomes

$$\sum_{i \in S_t} \delta_t(i) \nabla_{\theta} \log p_t(i) = \nabla_{\theta} \frac{1}{2} \sum_{i \in S_t} \delta_t(i)^2, \quad (5.32)$$

an unweighted squared log-ratio on S_t . It is still minimized at $p_t = q_t$ on S_t , but it is not a KL divergence. It also treats all k candidates equally, so tail candidates with small $p_t(i)$ and noisy log-ratios count as much as the head of the distribution.

¹Adding the constant Q_{S_t} , which does not change the gradient, gives $\Phi_t + Q_{S_t} = \sum_{i \in S_t} [p_t(i) \log(p_t(i)/q_t(i)) - p_t(i) + q_t(i)] \geq 0$, with equality iff $p_t = q_t$ on S_t . This is the generalized KL divergence between the unnormalized vectors $(p_t(i))_{i \in S_t}$ and $(q_t(i))_{i \in S_t}$.

5.2.2.2 Implementation Walkthrough

The computation flow inside `ppo/ray_trainer.py` is as follows. Each step computes one of the quantities defined in Section 5.2.2.1.

1. Generates the on-policy responses[↗] via

```
gen_batch_output = self.actor_rollout_wg.generate_sequences(gen_batch_output).
```

Here, `generate_sequences` produces the sampled student responses $y \sim \pi_\theta(\cdot | x)$ using the inference backend, e.g., vLLM or SGLang.

2. Recomputes the student log-probabilities[↗] `old_log_prob = self.actor_rollout_wg.compute_log_prob(batch)`. The training backend, e.g., FSDP, is used to compute the log-probabilities. The worker call eventually reaches `DataParallelPPOActor.compute_log_prob`[↗], which invokes `DataParallelPPOActor._forward_micro_batch`[↗]. When `top_k > 0`, the actor computes the student top- k token IDs and their log-probabilities:

```
1  # assume input_ids is (B, T) and attention_mask is (B, T)
2  # attention_mask is 1 for real tokens and 0 for padding.
3  # NOTE: the input_ids contain (prefix, response, special tokens)
4  # no generation is performed here.
5
6  input_ids = input_ids.unsqueeze(-1) # (B, T) -> (B, T, 1)
7
8  # unpad_input removes all positions for which attention_mask == 0
9  # example: input_ids = [ [11, 12], [21, 0]], attention_mask = [ [1, 1], [1, 0]]
10 # then after unpad:
11 # input_ids_rmpad = [ [11], [12], [21]], indices = [0, 1, 3], cu_seqlens = [0, 2, 3]
12
13 input_ids_rmpad, indices, cu_seqlens, *_ = unpad_input(input_ids, attention_mask)
14 input_ids_rmpad = input_ids_rmpad.transpose(0, 1) # (total_nnz, 1) -> (1, total_nnz)
15
16 # skip the details for
17 # 1. unpadding the position_ids
18 # 2. handling the multi-modal inputs, and
19 # 3. optionally Ulysses Sequence Parallelism
20
21 # compute the selected token ids by shifting the input_ids_rmpad to the right by
22 # 1 position. Example: input_ids_rmpad = [ [11], [12], [21]], then
23 # input_ids_rmpad_rolled = [ [12], [21], [11]]
24
25 input_ids_rmpad_rolled = torch.roll(input_ids_rmpad, shifts=-1, dims=1)
26 # (total_nnz, 1) -> (total_nnz,)
27 input_ids_rmpad_rolled = input_ids_rmpad_rolled.squeeze(0)
28
29 # compute the logits; prevent model thinks we are generating
30 output = self.actor_module(
31     input_ids=input_ids_rmpad,
32     attention_mask=None,
33     position_ids=position_ids_rmpad,
34     ...
35 )
36
37 # get the logits for the full vocabulary
38 logits_rmpad = output.logits.squeeze(0) # (total_nnz, vocab_size)
39 logits_rmpad.div_(temperature)
40
41 # compute the log-probabilities for the full vocabulary
42 log_probs_all = torch.log_softmax(logits_rmpad, dim=-1) # (total_nnz, vocab_size)
43 # Gather log_probs for target tokens
44 log_probs = log_probs_all.gather(
45     dim=-1, index=input_ids_rmpad_rolled.unsqueeze(-1)).squeeze(-1) # (total_nnz,)
46
47 # compute the top-k log-probabilities
48 _, topk_ids = torch.topk(logits_rmpad, k=top_k, dim=-1) # (total_nnz, top_k)
49
50 topk_log_probs = log_probs_all.gather(dim=-1, index=topk_ids) # (total_nnz, top_k)
```

```

51     full_topk_ids = pad_input(
52         hidden_states=topk_ids,
53         indices=indices,
54         batch=batch_size,
55         seqlen=seqlen,
56     )
57
58     full_topk_log_probs = pad_input(
59         hidden_states=topk_log_probs,
60         indices=indices,
61         batch=batch_size,
62         seqlen=seqlen,
63     )
64     # only return response part:
65     log_probs = full_log_probs.squeeze(-1)[: , -response_length - 1 : -1] # (bsz,
66         response_length)
67     topk_ids = full_topk_ids[: , -response_length - 1 : -1, :]
68     topk_log_probs = full_topk_log_probs[: , -response_length - 1 : -1, :]

```

Therefore, for each student-visited prefix $(x, y_{<t})$,

$$\text{student_top_k_ids} = S_t = \text{TopK}(\pi_\theta(\cdot \mid x, y_{<t})),$$

and

$$\text{student_top_k_log_probs}[i] = \log \pi_\theta(i \mid x, y_{<t}) \equiv \log p_t(i) \quad i \in S_t.$$

3. Then computes the [teacher log-probabilities](#) via the reward-model worker

```
teacher_data = self.rm_wg.compute_rm_score(batch)
```

Under the hood, it calls [RewardModelWorker.compute_rm_score](#), which invokes

[RewardModelWorker._forward_micro_batch](#), which is very similar to [DataParallelPPOActor._forward_micro_batch](#).

To highlight the core logic, we even use the most simple [branch](#), i.e, not removing padding and not using Ulysses Sequence Parallelism.

```

1  # the same sequence processing as in the actor forward pass
2  input_ids = micro_batch["input_ids"]
3  output = self.reward_module(
4      input_ids=input_ids,
5      attention_mask=attention_mask,
6      position_ids=position_ids,
7      ...
8  )
9  rm_output_logits = output.logits
10 # rm_logits_resp: (bsz, response_length, vocab_size)
11 rm_logits_resp = rm_output_logits[: , -response_length - 1 : -1, :]
12 rm_logits_resp = rm_logits_resp.div_(teacher_temperature)
13
14 # rm_log_probs: (bsz, response_length, vocab_size)
15 rm_log_probs = verl_F.logprobs_from_logits(
16     logits=rm_logits_resp, labels=micro_batch["responses"])
17
18 # compute teacher top k log-probabilities
19
20 # rm_logits_resp: (bsz, response_length, vocab_size)
21 flat_logits = rm_logits_resp.reshape(-1, rm_logits_resp.size(-1)) # (N, vocab_size)
22 # student_top_k_ids: (bsz, seqlen, K_s)
23 flat_ids = student_top_k_ids.reshape(-1, student_top_k_ids.size(-1)) # (N, K_s)
24
25 flat_on_student_log_probs, flat_counts, flat_overlap_mask,
26     flat_teacher_top_k_ids, flat_teacher_top_k_log_probs, flat_teacher_in_student =
27     self._compute_teacher_top_k_log_probs(
28         logits=flat_logits, student_ids=flat_ids, top_k=top_k, strategy=strategy)
29
30 # ... omit operations to get the tensors' shapes right
31

```

At this point, the last piece is on the mechanics of `_compute_teacher_top_k_log_probs`⁴. It offers 5 options for selecting the top-k token sets:

- `only_stu`: select top-k tokens from the student, then query the teacher for corresponding log-probs
- `only_tch`: select top-k tokens from the teacher, then query the student for corresponding log-probs
- `intersection`: keep tokens appearing in both student and teacher top-k sets
- `union`: merge student and teacher top-k sets
- `union-intersection`: tokens in either Top-K but not both, i.e. symmetric difference

```

1  # we only show the code for the default strategy: only_stu
2
3  # logits: (n_samples, Vocab)
4  # student_ids: (n_samples, K_s)
5  # output: (n_samples, K_s)
6
7  # use chunked top_k to save the memory
8  for start in range(0, n_samples, chunk_size):
9      end = min(start + chunk_size, n_samples)
10     logits_chunk = logits[start:end] # (chunk, Vocab)
11     student_ids_chunk = student_ids[start:end] # (chunk, K_s)
12
13     # Top-K on Teacher
14     # (chunk, K_t) and K_t = K_s = top_k
15     t_logits, t_ids = torch.topk(logits_chunk, k=top_k, dim=-1)
16     t_logsumexp = torch.logsumexp(logits_chunk, dim=-1, keepdim=True)
17     t_log_probs_top_k = t_logits - t_logsumexp
18
19     # ... omitted for brevity ...
20
21     chunk_log_probs = torch.gather(
22         logits_chunk, dim=-1, index=student_ids_chunk) - t_logsumexp
23
24
25

```

The gathered values are returned as `teacher_on_student_log_probs`, so for `only_stu`,

$$\text{teacher_on_student_log_probs}[i] = \log \pi_{\text{Teacher}}(i \mid x, y_{<t}) \equiv \log q_t(i) \quad i \in S_t,$$

with the teacher logits divided by `teacher_temperature`.

4. Once the student and teacher top-k log-probabilities are computed, the next step is to compute the reward. This is done by `compute_distillation_reward`⁵

`distillation_output = self.actor_rollout_wg.compute_distillation_reward(batch)`, which calls `DataParallelPPOActor.compute_distillation_reward`⁶. It also supports the above 5 strategies. We use the default strategy `only_stu` to demonstrate the computation.

```

1  S_logp = student_top_k_log_probs # (n_samples, top_k)
2  T_logp = teacher_top_k_log_probs # (n_samples, top_k)
3  T_on_S = teacher_on_student_log_probs # (n_samples, top_k)
4
5  # normalize the student log-probabilities on the top-k support
6  log_probs = S_logp
7  norm_log_weights = log_probs - torch.logsumexp(log_probs, dim=-1, keepdim=True)
8  weights = torch.exp(norm_log_weights)
9  norm_weights = torch.nan_to_num(weights, nan=0.0, posinf=0.0, neginf=0.0)
10 kl_val = S_logp - T_on_S
11 rm_scores = -kl_val * norm_weights
12

```

Here,

$$\text{norm_weights}_i = \frac{\pi_\theta(i \mid x, y_{<t})}{\sum_{j \in S_t} \pi_\theta(j \mid x, y_{<t})} \equiv \bar{p}_t^{(S_t)}(i) \text{ for } i \in S_t$$

under the default `reward_weight_mode="student_p"` (the alternatives `"teacher_p"` and `"none"` use teacher or uniform weights, and the analysis of Section 5.2.2.1 does not apply to them), and `kl_val` is $\delta_t(i)$. The produced candidate-level reward is

$$r_{t,i} = -\text{sg}\left[\bar{p}_t^{(S_t)}(i) \delta_t(i)\right] \quad \text{for } i \in S_t. \quad (5.33)$$

i.e., the per-candidate advantage (5.25) before masking. The stop-gradient is automatic: `S_logp` and `T_on_S` were computed in earlier forward passes and carry no gradient. Note that `rm_scores` itself is never differentiated. Summing it over the candidates gives the value $-\sum_{i \in S_t} \bar{p}_t^{(S_t)}(i) \delta_t(i)$, but differentiating that sum through $\bar{p}_t^{(S_t)}$ would not give g_t ; the gradient enters only through the PPO loss in the update step. Finally, `compute_reward_weights` has a `normalize=False` option (used by the `union-intersection` strategy), which uses the weights $p_t(i)$ instead of $\bar{p}_t^{(S_t)}(i)$; the factor $1/P_{S_t}$ in (5.31) then disappears and the gradient is exactly $\nabla_{\theta} \Phi_t$, with S_t replaced by that strategy's candidate set.

- Once the reward is computed, the next step is to compute the advantage. This is done by `compute_advantage`[Ⓔ], which ultimately calls `compute_token_reward_direct_advantage`[Ⓔ]

```

1  advantage = compute_advantage(
2      batch, # where you get the reward from
3      adv_estimator= "token_reward_direct" # switch for OPD only.
4      ...
5  )
6  # in side: compute_token_reward_direct_advantage, rm_scores is reshaped to
7  # (bsz, seqlen, top_k) and passed here as the token-level advantage
8  advantage = rm_scores * response_mask

```

So the reward is used directly as the advantage, with no return, baseline, or normalization. Writing $m_t \in \{0, 1\}$ for the response mask,

$$A_{t,i} = m_t r_{t,i} = -m_t \text{sg}\left[\bar{p}_t^{(S_t)}(i) \delta_t(i)\right] \quad \text{for } i \in S_t. \quad (5.34)$$

On response positions this is exactly (5.25).

- Lastly, the code reaches `self.actor_rollout_wg.update_actor`[Ⓔ], which is the entrypoint of the PPO update. If the FSDP backend is used, then the update is done by `self.actor.update_policy`[Ⓔ]. With a 3D advantage, the actor recomputes $\log p_t(i)$ for the stored candidate ids, `log_prob_for_loss = topk_log_probs`, and the `PPO loss`[Ⓔ] is applied per candidate and then summed over the candidates:

```

1  # log_prob, old_log_prob, advantages: (bsz, response_length, top_k)
2  ratio = torch.exp(log_prob - old_log_prob)
3  pg_losses1 = -advantages * ratio
4  pg_losses2 = -advantages * torch.clamp(ratio, 1 - cliprange_low, 1 + cliprange_high)
5  pg_losses = torch.maximum(pg_losses1, pg_losses2) # (+ dual-clip for A < 0)
6  pg_losses = torch.sum(pg_losses, dim=-1) # sum over the top_k candidates
7  pg_loss = agg_loss(pg_losses, response_mask, loss_agg_mode)
8

```

This is the candidate PPO loss (5.26), aggregated over response positions as specified by `loss_agg_mode`. The loop has the same structure as in Section 5.2.1.1 (`mini-batches × ppo_epochs`[Ⓔ], one optimizer step per mini-batch), with two differences. First, the advantage (5.34) is computed once per PPO iteration from stored log-probabilities, so later optimizer steps of the same iteration use a stale $\delta_t(i)$ and $\bar{p}_t^{(S_t)}(i)$, unlike upstream VeRL. Second, when there is a single mini-batch and `ppo_epochs=1` (the `on_policy` flag), the code sets `old_log_prob = log_prob_for_loss.detach()`, so $w_{t,i} \equiv 1$ by construction. At the first optimizer step of each PPO iteration (in particular, in the `on_policy` case), the gradient is therefore g_t of (5.28), as established in Section 5.2.2.1.

5.2.2.3 Other unvisited reference

Implementing On-Policy Distillation: Lessons from Building OPD in VeRL[Ⓔ]

Appendix A

Appendix

A.1 Glossary

A.1.1 Reward Model and Critic Model

Reward model. A model that scores a completed unit of behavior. In single-step LLM RL, the unit is usually the whole response y to a prompt x , and the reward model returns an outcome reward $r(x, y)$ only after the response is finished. It acts like the grader: the policy writes the answer first, then the reward model assigns the score being optimized.

In multi-step LLM RL, such as tool-use or agentic interaction, the completed unit may be an action span a_i , a full trajectory, or the final answer after several environment observations. The reward signal may therefore arrive after each completed action, as R_i , or only at the end of the episode. In either case, the reward model or verifier evaluates completed behavior, not a half-written prefix.

Critic model. A model that predicts the eventual return from the current partial state, usually written as a value function $V_\phi(s_t)$ over the prefix state s_t . Unlike the reward model, the critic does not grade the finished answer directly; it is a running estimate of what the future reward signal will imply for the current token position.

In single-step LLM RL, the critic looks at a partially generated response prefix and predicts the return induced by the final outcome reward $r(x, y)$, possibly together with token-level KL penalties or other shaping terms. In multi-step LLM RL, the critic evaluates states such as $s_{i,j}$ before generating token $a_{i,j}$, and predicts the return over future tokens, future action spans, environment observations, and later rewards. Thus the critic is a partial-credit forecaster for prefixes, while the reward model is the grader for completed behavior. The critic is trained by regression to a return target

A.1.2 How to train a Critic Model under the PPO framework

The critic V_ϕ is a separate network—typically the policy or reward model with the vocabulary head replaced by a scalar head—which emits one number per position in a single causal forward pass. It is defined by

$$V_\phi(s_t) = \mathbb{E}[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \mid s_t], \quad (\text{A.1})$$

i.e. the expected discounted reward from time t onward, inclusive of r_t .

A.1.2.1 TD residual

The temporal difference residual is defined as

$$\delta_t = \underbrace{r_t + \gamma V_\phi(s_{t+1})}_{\text{forecast after the step}} - \underbrace{V_\phi(s_t)}_{\text{forecast before the step}} \quad (\text{A.2})$$

Read V_ϕ as a live win-probability meter. Before step t you believed $V_\phi(s_t)$; after taking the step you have banked r_t and can read the meter again. The difference is how much your outlook improved or worsened *because of this step*. Positive δ_t means better news than expected. δ_t measures how badly the critic contradicts itself between adjacent steps. A perfect critic has $\mathbb{E}[\delta_t] = 0$ everywhere. So δ does double duty—its expectation being nonzero trains the critic, its realization trains the actor.

A.1.2.2 The n -step ladder and GAE

Everything about GAE rests on the following cancellation. Substitute (A.2) into a discounted sum of residuals, writing V_k for $V_\phi(s_k)$, and take $t = 0$, $T = 3$ for concreteness:

$$\begin{aligned} \gamma^0 \delta_0 &= r_0 & + \gamma V_1 & - V_0 \\ \gamma^1 \delta_1 &= \gamma r_1 & + \gamma^2 V_2 & - \gamma V_1 \\ \gamma^2 \delta_2 &= \gamma^2 r_2 & + \gamma^3 V_3 & - \gamma^2 V_2 \\ \gamma^3 \delta_3 &= \gamma^3 r_3 & + \gamma^4 V_4 & - \gamma^3 V_3 \end{aligned} \quad (\text{A.3})$$

Consider the value terms alone. The $+\gamma V_1$ of the first row cancels the $-\gamma V_1$ of the second; $+\gamma^2 V_2$ cancels $-\gamma^2 V_2$; $+\gamma^3 V_3$ cancels $-\gamma^3 V_3$. *Multiplying row ℓ by γ^ℓ is precisely what makes the powers line up*—this is the only reason

the cancellation works. Two terms survive: $\gamma^4 V_4$, which is zero by the terminal condition $V_\phi(s_{T+1}) = 0$, and $-V_0$, which has nothing to cancel against. In general,

$$\boxed{\sum_{\ell=0}^{T-t} \gamma^\ell \delta_{t+\ell} = \underbrace{\sum_{\ell=0}^{T-t} \gamma^\ell r_{t+\ell}}_{\text{actual discounted return}} - V_\phi(s_t).} \quad (\text{A.4})$$

In words: *the total surprise accumulated over a trajectory equals the outcome minus what you believed at the start*. The total is fixed; all GAE does is decide *when* along the trajectory that total gets booked.

Note carefully that (A.4) is not equation (A.8) below—it is that equation evaluated at $\lambda = 1$. The telescoping only goes through with weights γ^ℓ ; the general $(\gamma\lambda)^\ell$ case is built later.

Truncating (A.4) after n terms gives the **n -step advantage estimator**

$$A_t^{(n)} \equiv \sum_{\ell=0}^{n-1} \gamma^\ell \delta_{t+\ell}, \quad (\text{A.5})$$

which interpolates between two extremes. At $n = 1$, $A_t^{(1)} = \delta_t$: minimum variance (only one step of randomness enters) but maximum bias, since you trust the critic’s forecast at $t+1$ completely. At $n = T - t + 1$ the sum runs to the end and (A.4) makes it the Monte Carlo advantage $(\sum_\ell \gamma^\ell r_{t+\ell}) - V_\phi(s_t)$: unbiased, since the critic appears only as a constant baseline, but maximum variance, because every random event after step t is now folded in—including events the token had no influence over.

GAE takes an exponentially weighted average over the whole ladder rather than picking one rung:

$$A_t^{\text{GAE}} = (1 - \lambda) \sum_{n \geq 1} \lambda^{n-1} A_t^{(n)}. \quad (\text{A.6})$$

The weights $(1 - \lambda)\lambda^{n-1}$ sum to one, so this is a genuine average. Substituting (A.5) and exchanging the order of summation, the coefficient on $\delta_{t+\ell}$ is

$$(1 - \lambda) \gamma^\ell \sum_{n \geq \ell+1} \lambda^{n-1} = (1 - \lambda) \gamma^\ell \cdot \frac{\lambda^\ell}{1 - \lambda} = (\gamma\lambda)^\ell, \quad (\text{A.7})$$

which recovers the familiar closed form

$$\boxed{A_t^{\text{GAE}} = \sum_{\ell=0}^{T-t} (\gamma\lambda)^\ell \delta_{t+\ell}.} \quad (\text{A.8})$$

The parameter λ controls how far future TD residuals are allowed to flow back to the current token. At $\lambda = 0$, only the immediate residual δ_t is used, so the estimate is strongly bootstrapped from the critic: lower variance, but more bias if the critic is wrong. At $\lambda = 1$, every future residual is included with only the discount factor γ^ℓ , giving the Monte Carlo-style estimate: lower bias, but higher variance because all future randomness is credited backward. Intermediate values make this an exponential credit-assignment rule. In LLM RL, where one often sets $\gamma = 1$ to avoid washing out long completions, λ becomes the main knob deciding how much delayed reward from later tokens, later tool calls, or the final verifier should influence earlier token decisions.

A.1.2.3 The return target

The critic needs a regression target for (A.13) below. There is no unique choice; there is a ladder, indexed by how many real steps you take before letting the critic guess the remainder:

$$\hat{R}_t^{(n)} = \underbrace{r_t + \gamma r_{t+1} + \cdots + \gamma^{n-1} r_{t+n-1}}_{n \text{ steps of observed reward}} + \underbrace{\gamma A^{(n)} V_\phi(s_{t+n})}_{\text{critic guesses the tail}}. \quad (\text{A.9})$$

At $n = 1$ this is the plain TD target $r_t + \gamma V_\phi(s_{t+1})$; at $n = T - t + 1$ the bootstrap term vanishes and it is the full observed return.

The key observation is that *each rung of this ladder differs from the matching rung of the advantage ladder by exactly $V_\phi(s_t)$* . Subtracting $V_\phi(s_t)$ from (A.9) and telescoping exactly as in (A.3) yields $\hat{R}_t^{(n)} - V_\phi(s_t) = A_t^{(n)}$, i.e.

$$A_t^{(n)} = \hat{R}_t^{(n)} - V_\phi(s_t) \iff \hat{R}_t^{(n)} = A_t^{(n)} + V_\phi(s_t). \quad (\text{A.10})$$

Return targets and advantages are **the same object shifted by the baseline**—and crucially, the shift $V(s_t)$ is *the same for every rung*: it carries no n .

Apply the averaging operator $\sum_{n \geq 1} (1 - \lambda) \lambda^{n-1}(\cdot)$ to both sides. On the left it produces the λ -return by definition; on the right it splits in two, and because $V_\phi(s_t)$ is constant in n it factors straight out of the second sum:

$$\underbrace{\sum_{n \geq 1} (1 - \lambda) \lambda^{n-1} \hat{R}_t^{(n)}}_{\hat{R}_t^\lambda} = \underbrace{\sum_{n \geq 1} (1 - \lambda) \lambda^{n-1} A_t^{(n)}}_{A_t^{\text{GAE}} \text{ by (A.6)}} + V_\phi(s_t) \underbrace{\sum_{n \geq 1} (1 - \lambda) \lambda^{n-1}}_{= 1}. \quad (\text{A.11})$$

The leftover weight sum is exactly 1—this is where it matters that (A.6) is a genuine *average* and not an arbitrary linear combination—so the baseline passes through untouched and

$$\boxed{\hat{R}_t \equiv \hat{R}_t^\lambda = A_t^{\text{GAE}} + V_\phi(s_t)}. \quad (\text{A.12})$$

Note that no particular n has been selected here; the identity holds because *every* rung shares the same baseline. The bare $\hat{R}_t$ drops the λ superscript because this blended quantity is *the* target: the one equation (A.13) regresses against. So (A.12) is not a new estimator—it is a rearrangement. There is one blended estimate of how good s_t turned out to be. Viewed as a *deviation from the critic's prediction* it is the advantage that drives the actor; viewed as an *absolute number* it is the target the critic regresses toward:

$$\mathcal{L}^{\text{VF}}(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - \hat{R}_t)^2 \right]. \quad (\text{A.13})$$

Informally: $V_\phi(s_t)$ is the prior, A_t^{GAE} is the accumulated (γ -discounted, λ -decayed) news that arrived afterward, and the sum is the posterior estimate you now wish the critic had produced.

So is the critic minimizing the advantage?

Substituting (A.12) into (A.13) gives $V_\phi(s_t) - \hat{R}_t = -A_t^{\text{GAE}}$, hence

$$\mathcal{L}^{\text{VF}}(\phi) = \mathbb{E}_t \left[(A_t^{\text{GAE}})^2 \right]. \quad (\text{A.14})$$

The critic loss *is* the squared advantage. Two qualifications keep this from misleading.

The target is detached. $\hat{R}_t$ is computed once from the *old* critic, before the optimization epochs, and frozen; gradients flow only through the fresh $V_\phi(s_t)$ on the left. So (A.14) holds numerically at the first inner epoch, where $\phi = \phi_{\text{old}}$, and thereafter the loss is a regression against a stale constant.

```

1  with torch.no_grad():                                # <-- everything here is frozen
2      values      = critic(states)                      # V_{phi_old}
3      advantages  = compute_gae(rewards, values, gamma, lambda)
4      returns     = advantages + values                 # hat{R}, a constant tensor
5
6  for epoch in range(K):                                # PPO inner epochs
7      v = critic(states)                                # only THIS carries gradient
8      vf_loss = ((v - returns) ** 2).mean()
```

What matters is the gradient,

$$\nabla_\phi \mathcal{L}^{\text{VF}} = 2 \mathbb{E}_t \left[(V_\phi(s_t) - \hat{R}_t) \nabla_\phi V_\phi(s_t) \right] = -2 \mathbb{E}_t \left[A_t^{\text{GAE}} \nabla_\phi V_\phi(s_t) \right], \quad (\text{A.15})$$

which says: *the advantage is the critic's error signal*. Positive advantage means the state turned out better than predicted, so raise $V_\phi(s_t)$ by an amount proportional to $|A_t|$. This is exactly the forward view of TD(λ), $\Delta\phi \propto (\hat{R}_t^\lambda - V_\phi(s_t)) \nabla_\phi V_\phi(s_t)$, rewritten as a squared error so that autograd performs the update. Letting gradients flow through the bootstrapped $V_\phi(s_{t+n})$ inside $\hat{R}_t$ instead yields Baird's residual-gradient method—a different algorithm with a different fixed point, which invites the critic to reduce loss by dragging $V_\phi(s_{t+n})$ toward $V_\phi(s_t)$, satisfying self-consistency by chasing its own tail rather than by becoming accurate. Semi-gradient (detached target) is standard. At $\lambda = 1$ the distinction is moot, since $\hat{R}_t$ is the observed return and contains no ϕ .

Minimizing $\mathbb{E}[A^2]$ does not drive A to zero. The minimizer of a squared error is the conditional mean, $V^*(s) = \mathbb{E}[\hat{R}_t \mid s_t = s]$, and the residual there is the conditional variance $\text{Var}(\hat{R}_t \mid s_t)$ —irreducible however good the critic becomes. A perfect critic yields $\mathbb{E}[A_t \mid s_t] = 0$, not $A_t = 0$.

There is a structural reason it cannot collapse: $V_\phi(s_t)$ never sees the action a_t . It is evaluated on the prefix *before* the token is sampled, so it can represent only the average over actions under the current policy. The action-specific deviation $A_t = Q(s_t, a_t) - V_\phi(s_t)$ is precisely the component the critic is architecturally forbidden to absorb. It strips out what is predictable from the state and leaves untouched the question of which token was actually chosen.

A.1.2.4 Implementation notes

Backward recursion. Equation (A.8) is never evaluated as a sum. Expanding it one term gives the recursion actually used,

$$A_t = \delta_t + \gamma \lambda A_{t+1}, \quad A_{T+1} = 0, \quad (\text{A.16})$$

computed in a single backward pass—“my advantage is my own surprise, plus a discounted share of everything that surprises the future.” Equation (A.12) is then one addition, `returns = advantages + values`, which is the real reason the target is written in that form.

Normalization ordering. Advantages are usually normalized to zero mean and unit variance before the policy loss. The return targets must be computed *before* that normalization, or the critic is trained toward a rescaled quantity. This is a common and quiet bug.

Discounting at token granularity. A 20k-token rollout with $\gamma = 0.99$ discounts the terminal reward by roughly e^{-200} , annihilating the signal. Hence $\gamma = 1$ is effectively universal in LLM RL, which leaves λ carrying the entire bias–variance tradeoff by itself.

Masking observations. Failing to mask environment tokens out of the policy loss trains the model to *predict tool outputs*, so it learns to hallucinate plausible observations and answer from them. The environment gets distilled into the policy and rewarded for confabulating.

Stochastic transitions at tool boundaries. In single-turn LLM RL the transition is deterministic— s_{t+1} is s_t with your own token appended—so all randomness lives in the policy. At a tool boundary the transition is genuinely stochastic: the same query may return an excellent document or garbage. The residual across that boundary, $\delta_t = \gamma V_\phi(s_t \oplus \text{obs}) - V_\phi(s_t)$, mixes skill (a well-formed query) with luck (a cooperative index), and PPO cannot separate them. At $\lambda = 1$ that luck propagates undiminished back onto every earlier token, which is a concrete argument for $\lambda < 1$ —or for turn-level value baselines—in agentic training.

Bibliography

- Agarwal, R., N. Vieillard, Y. Zhou, P. Stanczyk, S. Ramos Garea, M. Geist, and O. Bachem (2024). “On-policy distillation of language models: Learning from self-generated mistakes”. In: *International Conference on Learning Representations*. Vol. 2024, pp. 21246–21263.
- Chen, A., A. Li, B. Gong, et al. (2025). *MiniMax-M1: Scaling Test-Time Compute Efficiently with Lightning Attention*. arXiv: 2506.13585 [cs.CL] ^u. URL: <https://arxiv.org/abs/2506.13585>.
- Hilton, J., K. Cobbe, and J. Schulman (2022). *Batch size-invariance for policy optimization*. arXiv: 2110.00641 [cs.LG] ^u. URL: <https://arxiv.org/abs/2110.00641>.
- Hou, Z., Y. Li, J. Tang, and Y. Dong (2026). “Single-Rollout Asynchronous Optimization for Agentic Reinforcement Learning”. In: *arXiv preprint arXiv:2607.07508*.
- Li, Y., Z. Hou, Y. Jing, J. Tang, and Y. Dong (2026). *CompactionRL: Reinforcement Learning with Context Compaction for Long-Horizon Agents*. arXiv: 2607.05378 [cs.LG] ^u. URL: <https://arxiv.org/abs/2607.05378>.
- Lu, M., W. Sun, W. Du, Z. Ling, X. Yao, K. Liu, and J. Chen (2025). *Scaling LLM Multi-turn RL with End-to-end Summarization-based Context Management*. arXiv: 2510.06727 [cs.CL] ^u. URL: <https://arxiv.org/abs/2510.06727>.
- Ouyang, L., J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. (2022). “Training language models to follow instructions with human feedback”. In: *Advances in neural information processing systems* 35, pp. 27730–27744.
- Schulman, J., P. Moritz, S. Levine, M. Jordan, and P. Abbeel (2015). “High-dimensional continuous control using generalized advantage estimation”. In: *arXiv preprint arXiv:1506.02438*.
- Xie, T., Z. Gao, Q. Ren, H. Luo, Y. Hong, B. Dai, J. Zhou, K. Qiu, Z. Wu, and C. Luo (2025). “Logic-RL: Unleashing LLM Reasoning with Rule-Based Reinforcement Learning”. In: *arXiv preprint arXiv:2502.14768*.